

RNN++

Yu LI (李煜)

Wednesday, 28 October 2026

liyu95.com

liyu@cse.cuhk.edu.hk

Department of Computer Science and Engineering (CSE)

The Chinese University of Hong Kong (CUHK)

Comments and questions



- Good
- good
- NA
- I need more time to understand this topic
- please correct the rules of kaggle2 to allow loading pretrained weights
  - Yes, its allowed
- good, very clear

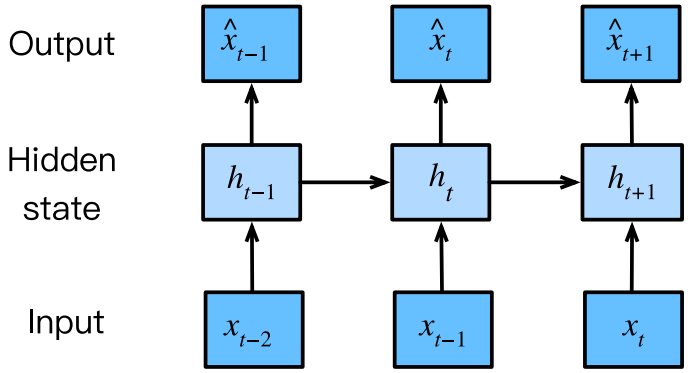
On tutorial lecture



- NA
- Could maybe cover more of the top performing models and how they differ etc.
- provide exhibition of top ranking code of kaggle 1 on the course website
  - **We will ask whether they are willing to share. If so, we will post the code.**

Hidden state model

- Classification?
  - Use the last hidden state to predict



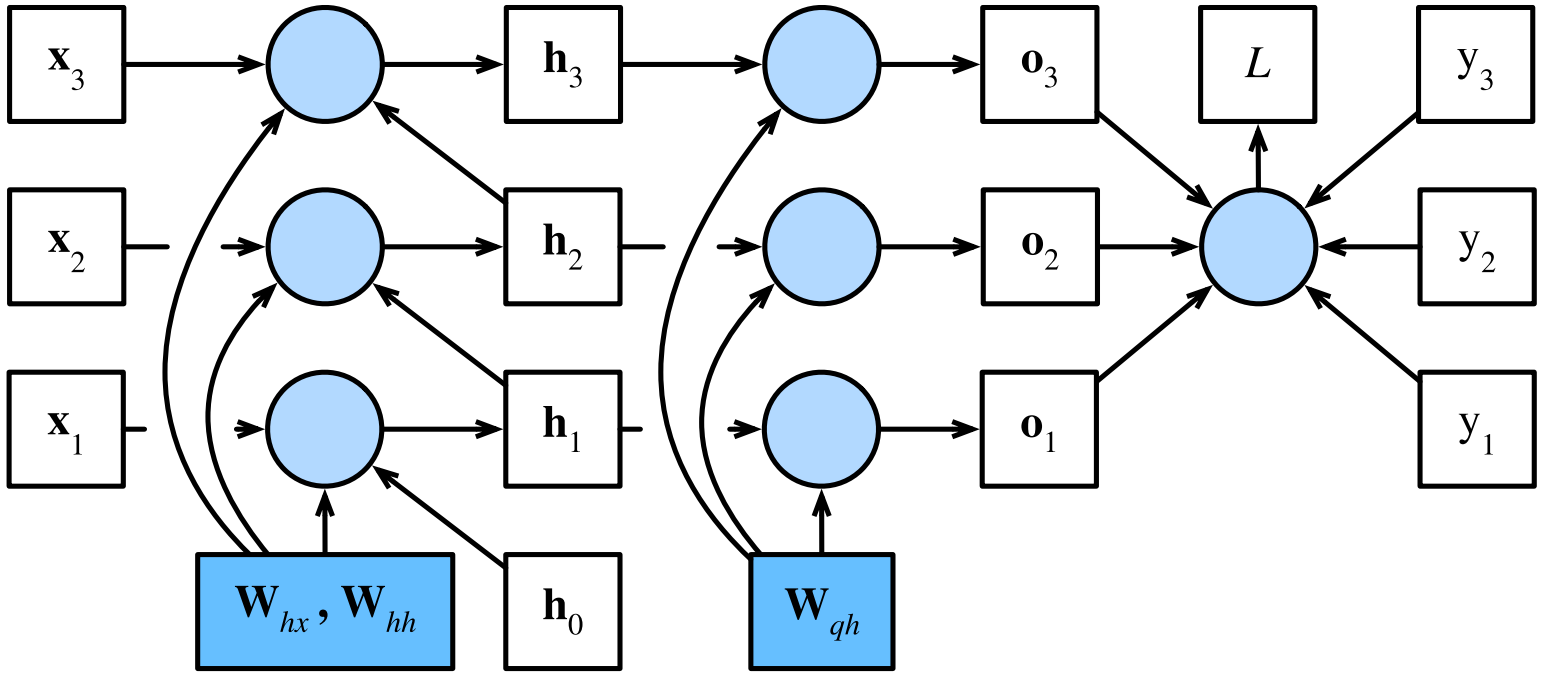
- Regression?
  - Can also predict at each position/timepoint
  - Based on the previous values



- Autoregressive models
  - As long as based on the previous values
  - No matter with hidden state or not

Train RNNs with GD

- Initialize all the weights
  - Random values
- For all the training data
  - Forward pass
  - Calculate the output Y
- Update all the weights
  - $w_{ij}^k = w_{ij}^k - \Delta w_{ij}^k$
  - $\Delta w_{ij}^k = \alpha \frac{\partial Div(Y,d)}{\partial w_{ij}^{(k)}}$
  - α is a small constant
- Repeat the above step
  - Until no more to update



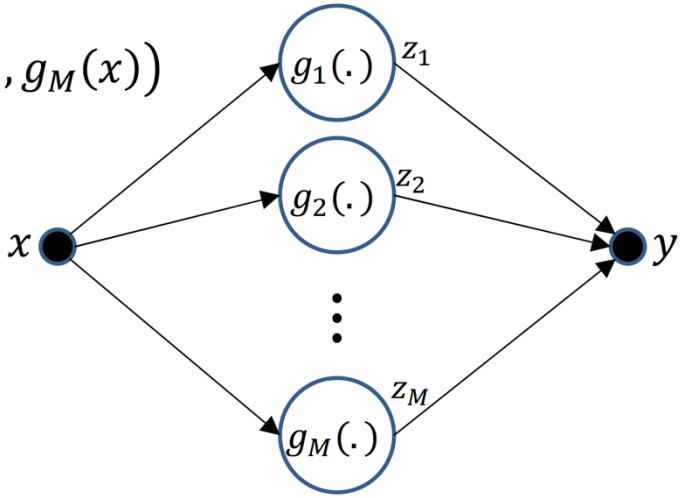
How many layers are in this network?

Calculus refresher: Chain rule



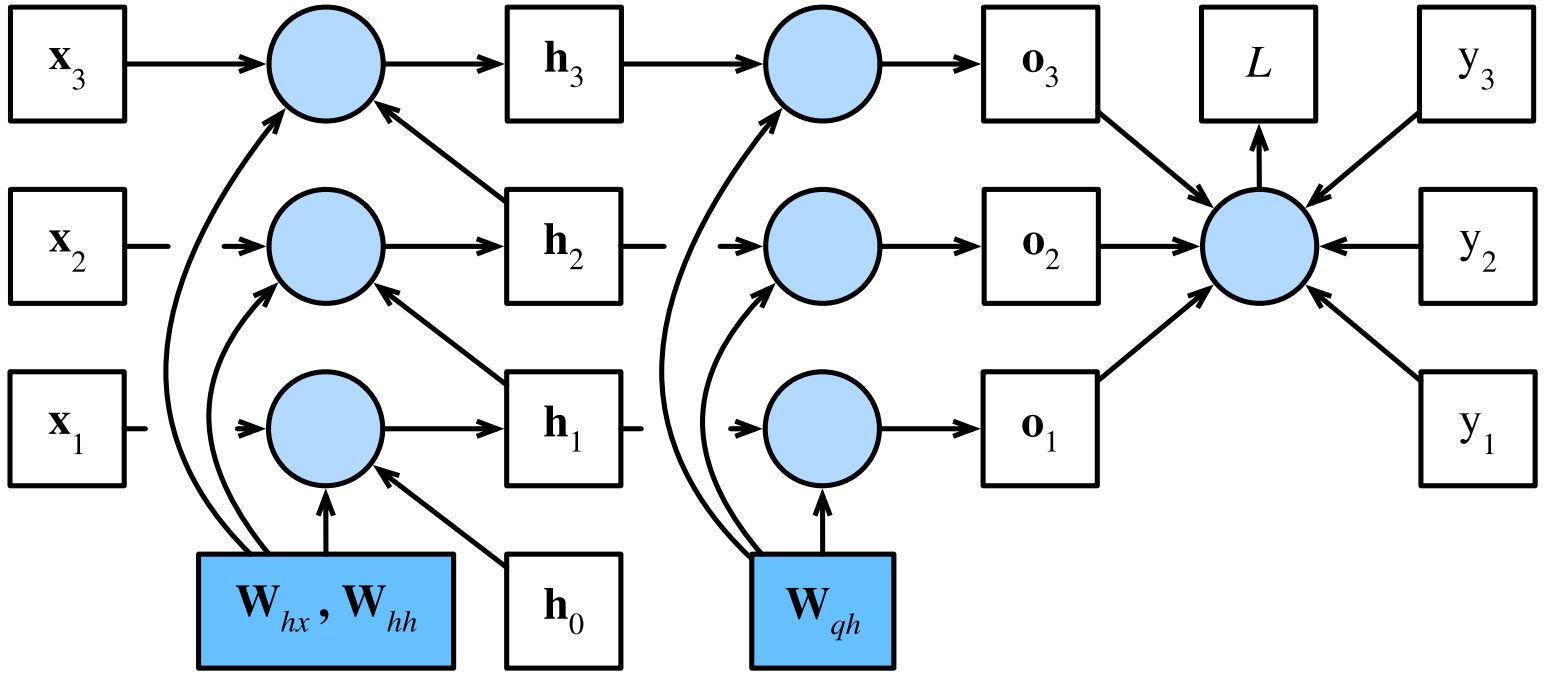
- For any nested function: $y = f(g(x))$
  - $\frac{dy}{dx} = \frac{df}{dg(x)}\frac{dg(x)}{dx}$

$y = f(g_1(x), g_1(x), \dots, g_M(x))$



- For the distributed nested function
  - $y = f(g_1(x), g_2(x), \dots, g_M(x))$
  - x affects y through each of g_1, ..., g_M
  - Small perturbations in x cause small perturbations in each of g_1, ..., g_M, each of which individually additively perturbs y
  - $\frac{dy}{dx} = \frac{dy}{dz_1}\frac{dz_1}{dx} + \frac{dy}{dz_2}\frac{dz_2}{dx} + \cdots + \frac{dy}{dz_M}\frac{dz_M}{dx}$

How to train RNNs?



Still use back-propagation to get the gradients and the optimization algorithms to update the weights

But the dependency is tricky in RNNs

Back-propagation on RNNs

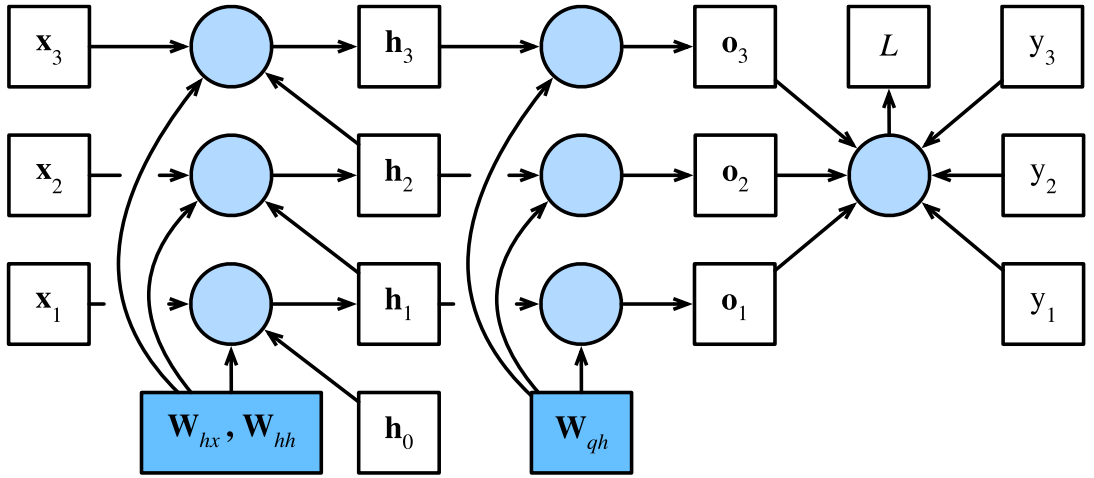


- $h_t = f(x_t, h_{t-1}, w_h)$
- $o_t = g(h_t, w_o)$
- {…, $(x_{t-1}, h_{t-1}, o_{t-1})$, (x_t, h_t, o_t), …}, depends on each other recursively
- $L(x_1, \dots, x_T, y_1, \dots, y_T, w_h, w_o) = \frac{1}{T}\sum_{t=1}^{T} l(y_t, o_t)$

- $\frac{\partial L}{\partial w_h} = \frac{1}{T}\sum_{t=1}^{T} \frac{\partial l(y_t,o_t)}{\partial w_h} = \frac{1}{T}\sum_{t=1}^{T} \frac{\partial l(y_t,o_t)}{\partial o_t}\frac{\partial g(h_t,w_o)}{\partial h_t}\frac{\partial h_t}{\partial w_h}$
- $\frac{\partial h_t}{\partial w_h} = \frac{\partial f(x_t,h_{t-1},w_h)}{\partial w_h} + \frac{\partial f(x_t,h_{t-1},w_h)}{\partial h_{t-1}}\frac{\partial h_{t-1}}{\partial w_h}$
- $\frac{\partial h_t}{\partial w_h} = \frac{\partial f(x_t,h_{t-1},w_h)}{\partial w_h} + \sum_{i=1}^{t-1}\left(\prod_{j=i+1}^{t} \frac{\partial f(x_j,h_{j-1},w_h)}{\partial h_{j-1}}\right)\frac{\partial f(x_i,h_{i-1},w_h)}{\partial w_h}$

Very long chain, explosion or vanishing

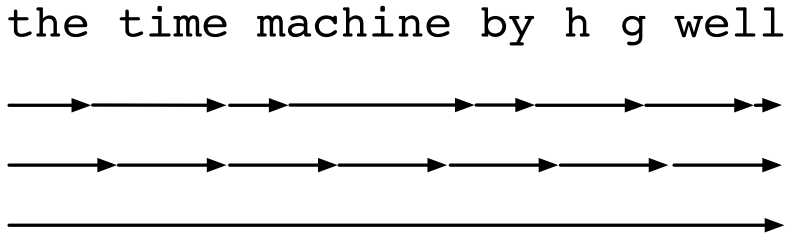
A real example



- $\boldsymbol{h}_t = \boldsymbol{W}_{hx}\boldsymbol{x}_t + \boldsymbol{W}_{hh}\boldsymbol{h}_{t-1}$
- $\boldsymbol{o}_t = \boldsymbol{W}_{qh}\boldsymbol{h}_t$
- $L = \frac{1}{T}\sum_{t=1}^{T} l(\boldsymbol{y}_t, \boldsymbol{o}_t)$
- $\frac{\partial L}{\partial \boldsymbol{h}_t} = \sum_{i=t}^{T} \left(\boldsymbol{W}_{\mathrm{hh}}^{\mathrm{T}}\right)^{T-i} \boldsymbol{W}_{\mathrm{qh}}^{\mathrm{T}} \frac{\partial L}{\partial \boldsymbol{o}_{T+t-i}}$
- $\frac{\partial L}{\partial \boldsymbol{W}_{hx}} = \sum_{t=1}^{T} prod\left(\frac{\partial L}{\partial \boldsymbol{h}_t}, \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{W}_{hx}}\right) = \sum_{t=1}^{T} \frac{\partial L}{\partial \boldsymbol{h}_t} \boldsymbol{x}_t^{\mathrm{T}}$
- $\frac{\partial L}{\partial \boldsymbol{W}_{hh}} = \sum_{t=1}^{T} prod\left(\frac{\partial L}{\partial \boldsymbol{h}_t}, \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{W}_{hh}}\right) = \sum_{t=1}^{T} \frac{\partial L}{\partial \boldsymbol{h}_t} \boldsymbol{h}_{t-1}^{\mathrm{T}}$
- Slow, explosion or vanishing

Truncation

- $\frac{\partial L}{\partial \boldsymbol{h}_t} = \sum_{i=t}^{T} \left(\boldsymbol{W}_{\mathrm{hh}}^{\mathrm{T}}\right)^{T-i} \boldsymbol{W}_{\mathrm{qh}}^{\mathrm{T}} \frac{\partial L}{\partial \boldsymbol{o}_{T+t-i}}$
- $\frac{\partial L}{\partial \boldsymbol{W}_{hx}} = \sum_{t=1}^{T} prod\left(\frac{\partial L}{\partial \boldsymbol{h}_t}, \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{W}_{hx}}\right) = \sum_{t=1}^{T} \frac{\partial L}{\partial \boldsymbol{h}_t} \boldsymbol{x}_t^{\mathrm{T}}$
- $\frac{\partial L}{\partial \boldsymbol{W}_{hh}} = \sum_{t=1}^{T} prod\left(\frac{\partial L}{\partial \boldsymbol{h}_t}, \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{W}_{hh}}\right) = \sum_{t=1}^{T} \frac{\partial L}{\partial \boldsymbol{h}_t} \boldsymbol{h}_{t-1}^{\mathrm{T}}$
- Full computation
  - Slow, explosion or vanishing
- Truncating time steps
  - Truncate the sum after τ times
- Randomized truncation
  - Truncate the sum after random t times



Today's agenda



- RNN++
  - GRU
  - LSTM
- Other models
  - Deep RNNs
  - Bidirectional model
  - Sequence to sequence model
    - Machine translation
- Attention

More prior knowledge



- Text can be long
  - A *recurrent hidden cell*: learn the pattern

- What else??

More prior knowledge



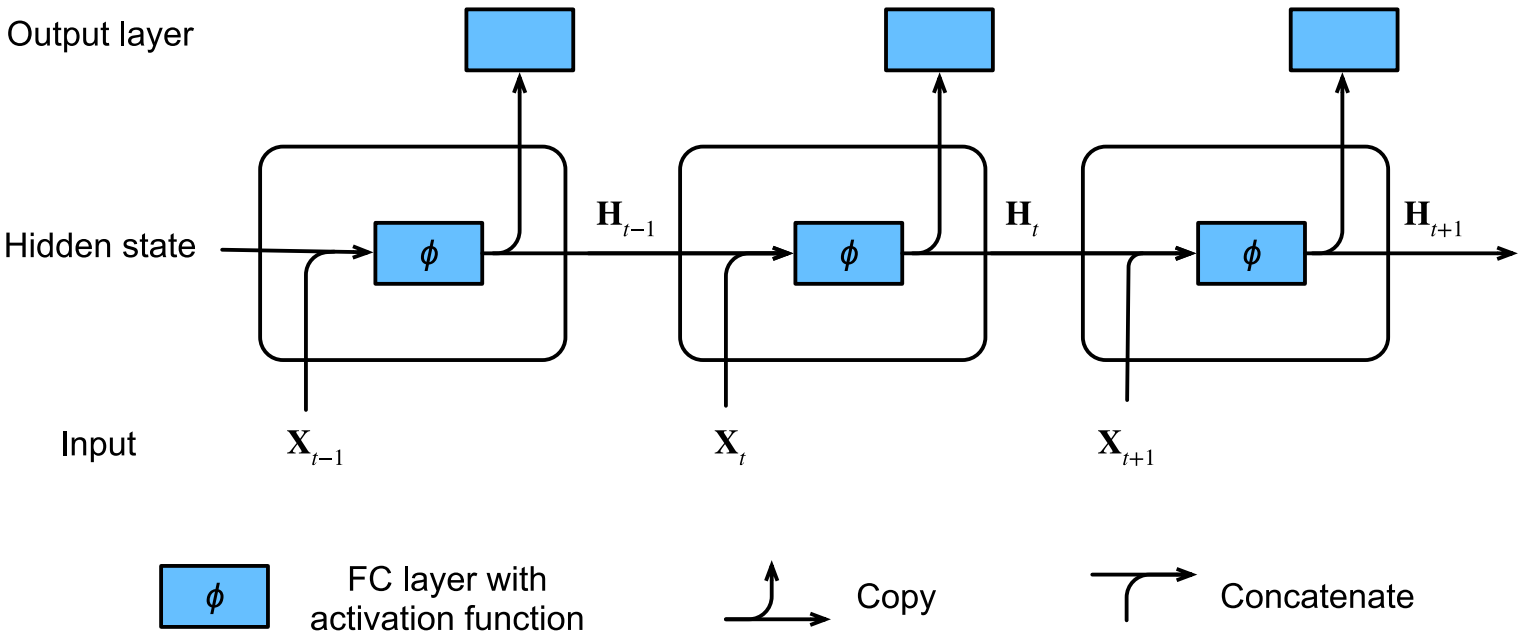
- Text can be long
  - A *recurrent hidden cell*: learn the pattern
- Early observation may be very important
  - "Yu is teaching AIST-4010, …(5000 words)…, wish you have a nice day!"
  - Question: who is teaching AIST-4010
  - Truncation may miss such information. It may also decay in the long sequence
  - A *memory cell*: remember important information
- Some information is meaningless
  - "
Homework (42%): One non-grading assignment (6%, set up the environment) and three grading homework (12%+12%+12%).
"
  - HTML tags
  - *Skip* such meaningless tokens

More prior knowledge



- Text can be long
  - A *recurrent hidden cell*: learn the pattern
- Early observation may be very important
  - Truncation may miss such information. It may also decay in the long sequence
  - A *memory cell*: remember important information
- Some information is meaningless
  - HTML tags
  - *Skip* such meaningless tokens
- Logic break
  - A new chapter
  - *Reset* the hidden state

Modify RNNs

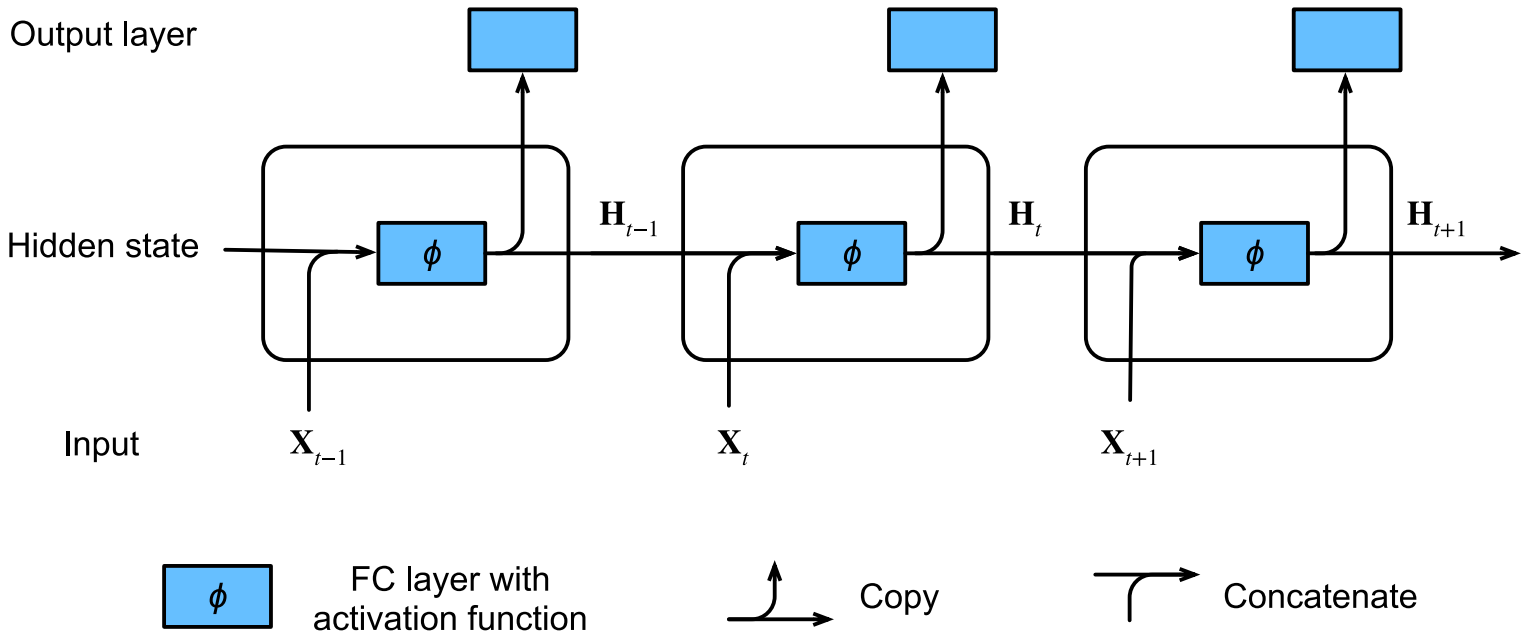


Memorize

Skip

Reset

Modify RNNs



Memorize

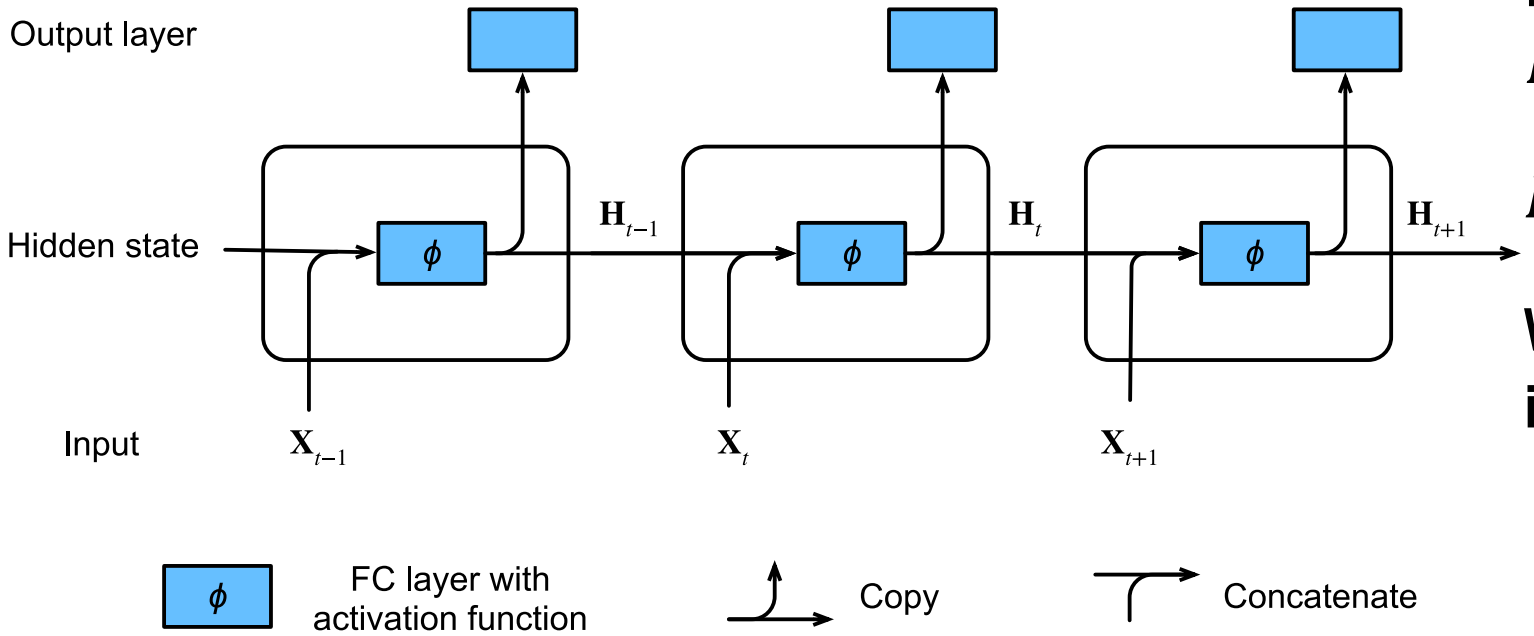
Skip

Reset

How much we use $\boldsymbol{X}_t$ and $\boldsymbol{H}_{t-1}$ to get $\boldsymbol{H}_t$

$$\boldsymbol{H}_t = \phi(\boldsymbol{X}_t \boldsymbol{W}_{xh} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hh} + \boldsymbol{b}_h)$$

Modify RNNs



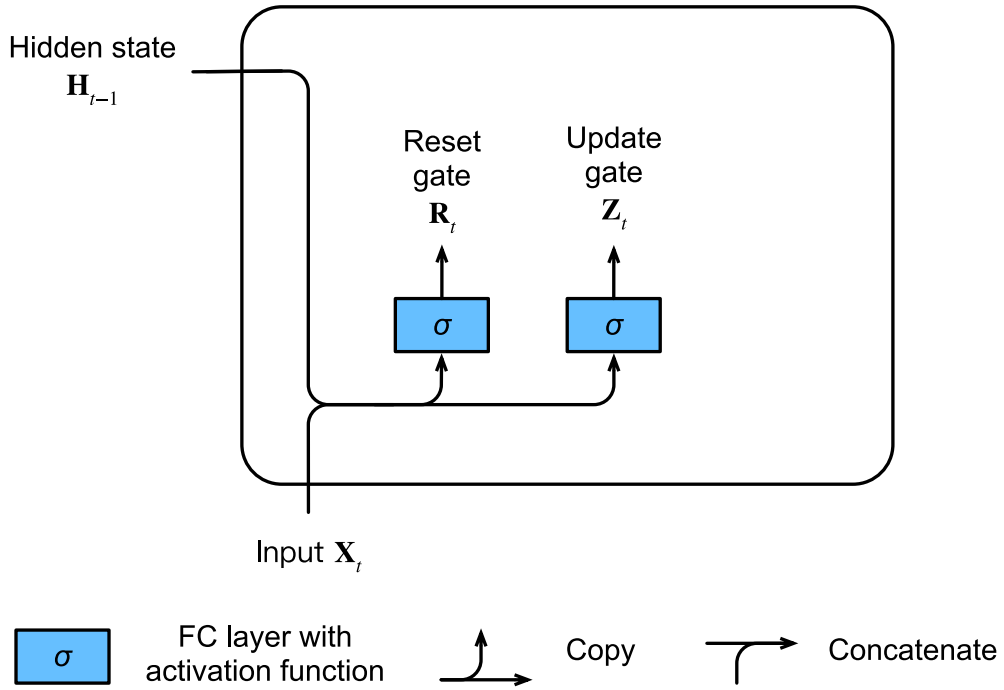
Memorize, Skip, Reset

How much we use $\boldsymbol{X}_t$ and $\boldsymbol{H}_{t-1}$ to get $\boldsymbol{H}_t$

$$\boldsymbol{H}_t = \phi(\boldsymbol{X}_t\boldsymbol{W}_{xh} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{hh} + \boldsymbol{b}_h)$$

We need some gates to turn the information on or off

Some gates



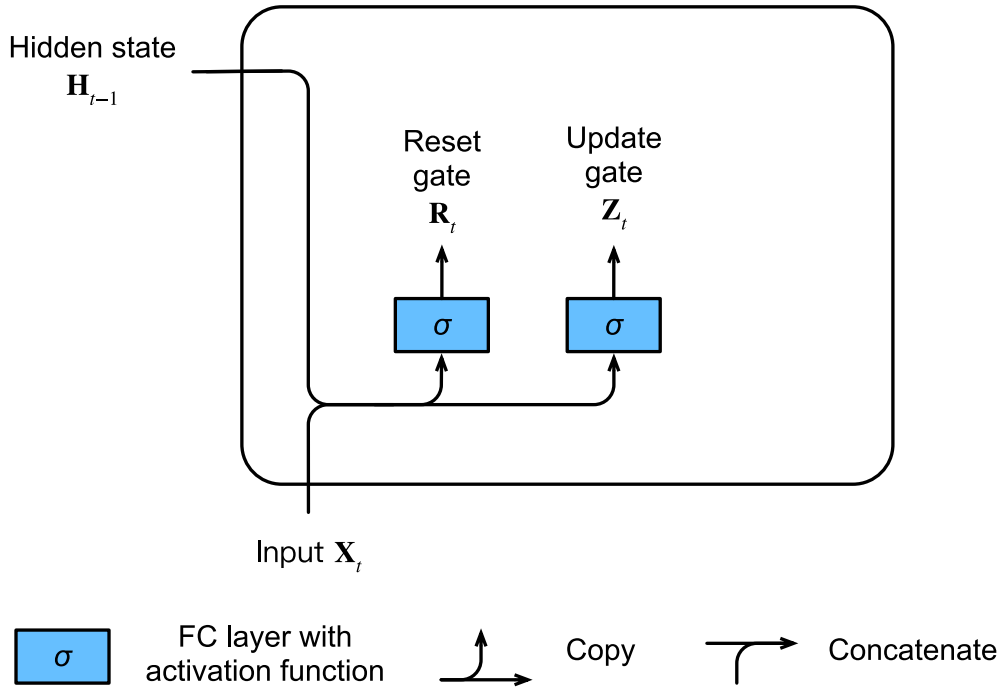
Memorize, Skip, Reset

How much we use $\boldsymbol{X}_t$ and $\boldsymbol{H}_{t-1}$ to get $\boldsymbol{H}_t$

$$\boldsymbol{H}_t = \phi(\boldsymbol{X}_t \boldsymbol{W}_{xh} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hh} + \boldsymbol{b}_h)$$

We need some gates to turn the information on or off

First attempt



Memorize, Skip, Reset

How much we use $\boldsymbol{X}_t$ and $\boldsymbol{H}_{t-1}$ to get $\boldsymbol{H}_t$

$$\boldsymbol{H}_t = \phi(\boldsymbol{X}_t\boldsymbol{W}_{xh} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{hh} + \boldsymbol{b}_h)$$

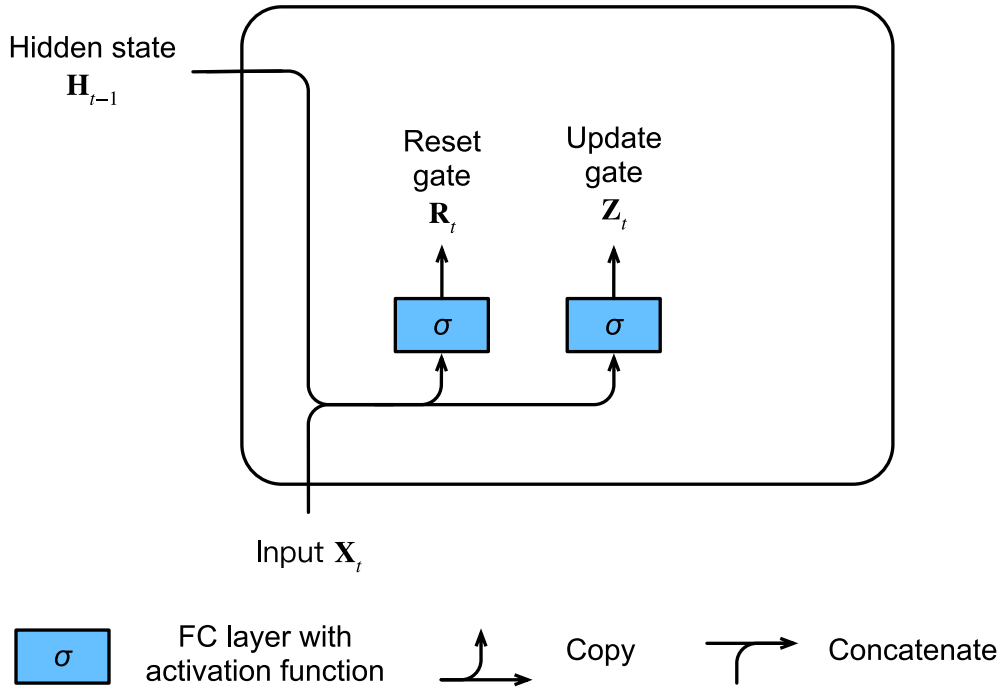
We need some gates to turn the information on or off

$$\boldsymbol{H}_t = \phi(\boldsymbol{X}_t\boldsymbol{W}_{xh} + (\boldsymbol{R}_t \odot \boldsymbol{H}_{t-1})\boldsymbol{W}_{hh} + \boldsymbol{b}_h)$$

$\boldsymbol{R}_t$: Binary vector

What function can we achieve?

Second attempt



Memorize, Skip, Reset

How much we use $\boldsymbol{X}_t$ and $\boldsymbol{H}_{t-1}$ to get $\boldsymbol{H}_t$

$$\boldsymbol{H}_t = \phi(\boldsymbol{X}_t\boldsymbol{W}_{xh} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{hh} + \boldsymbol{b}_h)$$

We need some gates to turn the information on or off

$$\boldsymbol{H}_t = \phi\big((\boldsymbol{Z}_t \odot \boldsymbol{X}_t)\boldsymbol{W}_{xh} + (\boldsymbol{R}_t \odot \boldsymbol{H}_{t-1})\boldsymbol{W}_{hh} + \boldsymbol{b}_h\big)$$

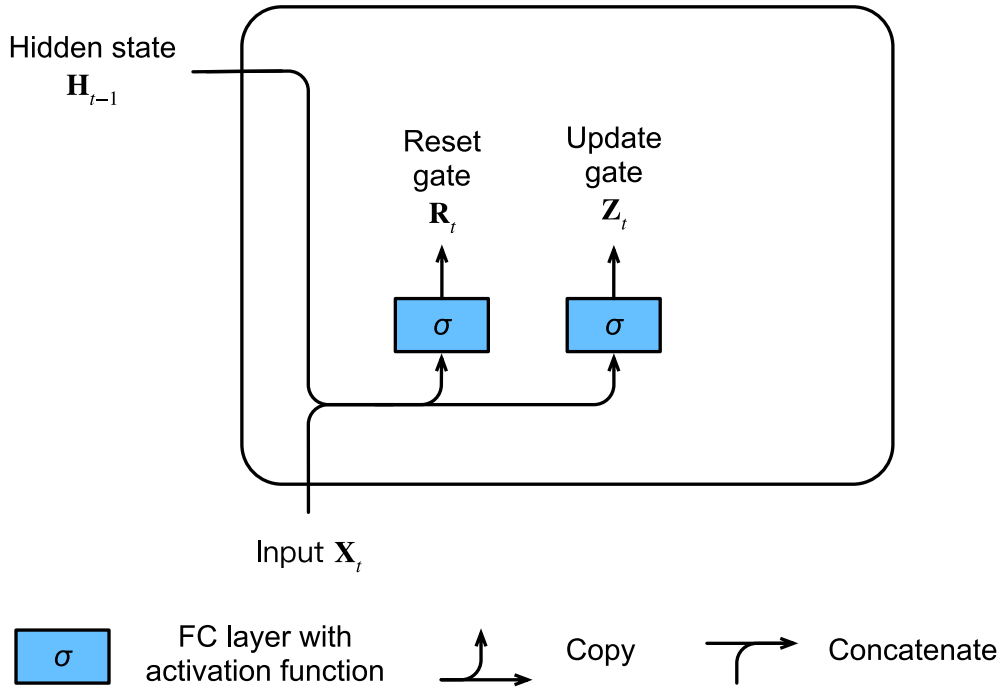
$\boldsymbol{R}_t$: Binary vector

$\boldsymbol{Z}_t$: Binary vector

What function can we achieve?

Any problem here?

Second attempt



Memorize, Skip, Reset

How much we use $\boldsymbol{X}_t$ and $\boldsymbol{H}_{t-1}$ to get $\boldsymbol{H}_t$

$$\boldsymbol{H}_t = \phi(\boldsymbol{X}_t\boldsymbol{W}_{xh} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{hh} + \boldsymbol{b}_h)$$

We need some gates to turn the information on or off

$$\boldsymbol{H}_t = \phi\big((\boldsymbol{Z}_t \odot \boldsymbol{X}_t)\boldsymbol{W}_{xh} + (\boldsymbol{R}_t \odot \boldsymbol{H}_{t-1})\boldsymbol{W}_{hh} + \boldsymbol{b}_h\big)$$

What if both gates are closed?

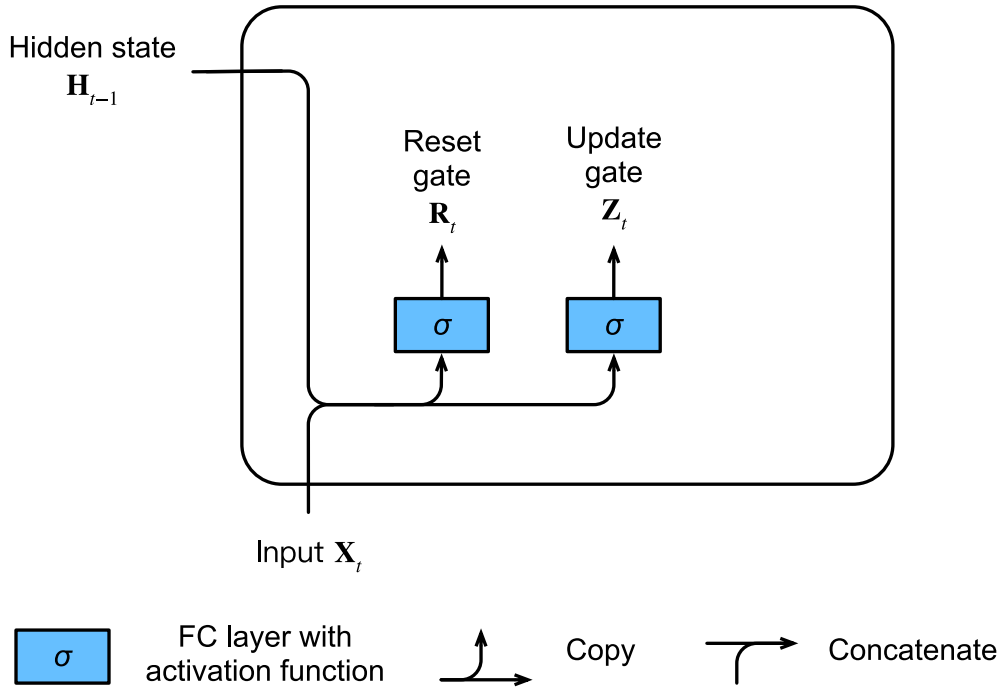
$\boldsymbol{R}_t$: Binary vector

$\boldsymbol{Z}_t$: Binary vector

What function can we achieve?

Any problem here?

Third attempt



Memorize, Skip, Reset

How much we use $\boldsymbol{X}_t$ and $\boldsymbol{H}_{t-1}$ to get $\boldsymbol{H}_t$

$$\boldsymbol{H}_t = \phi(\boldsymbol{X}_t\boldsymbol{W}_{xh} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{hh} + \boldsymbol{b}_h)$$

We need some gates to turn the information on or off

$$\boldsymbol{H}_t = \phi\big((\boldsymbol{Z}_t \odot \boldsymbol{X}_t)\boldsymbol{W}_{xh} + (\boldsymbol{R}_t \odot \boldsymbol{H}_{t-1})\boldsymbol{W}_{hh} + \boldsymbol{b}_h\big) + \boldsymbol{P}_t \odot \boldsymbol{H}_{t-1}$$

$\boldsymbol{R}_t$: Binary vector

$\boldsymbol{Z}_t$: Binary vector

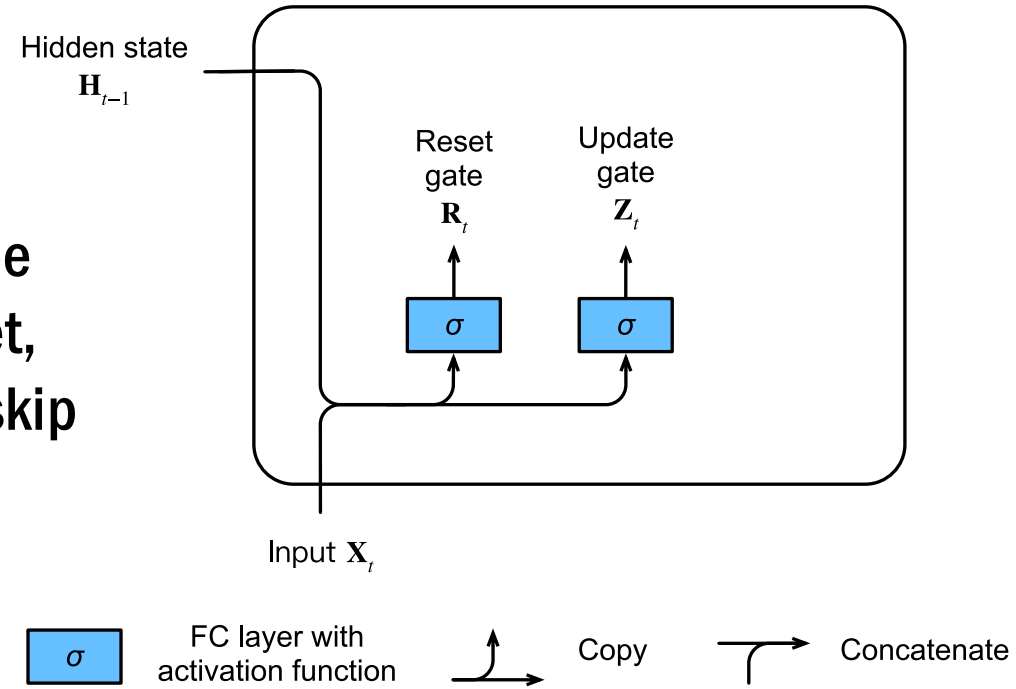
$\boldsymbol{P}_t$: Binary vector

What function can we achieve?

Any problem here?

Third attempt

Memorize should be exclusive from reset, and together with skip



Memorize, Skip, Reset

How much we use $\boldsymbol{X}_t$ and $\boldsymbol{H}_{t-1}$ to get $\boldsymbol{H}_t$

$$\boldsymbol{H}_t = \phi(\boldsymbol{X}_t\boldsymbol{W}_{xh} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{hh} + \boldsymbol{b}_h)$$

We need some gates to turn the information on or off

$$\boldsymbol{H}_t = \phi\big((\boldsymbol{Z}_t \odot \boldsymbol{X}_t)\boldsymbol{W}_{xh} + (\boldsymbol{R}_t \odot \boldsymbol{H}_{t-1})\boldsymbol{W}_{hh} + \boldsymbol{b}_h\big) + \boldsymbol{P}_t \odot \boldsymbol{H}_{t-1}$$

$\boldsymbol{R}_t$: Binary vector

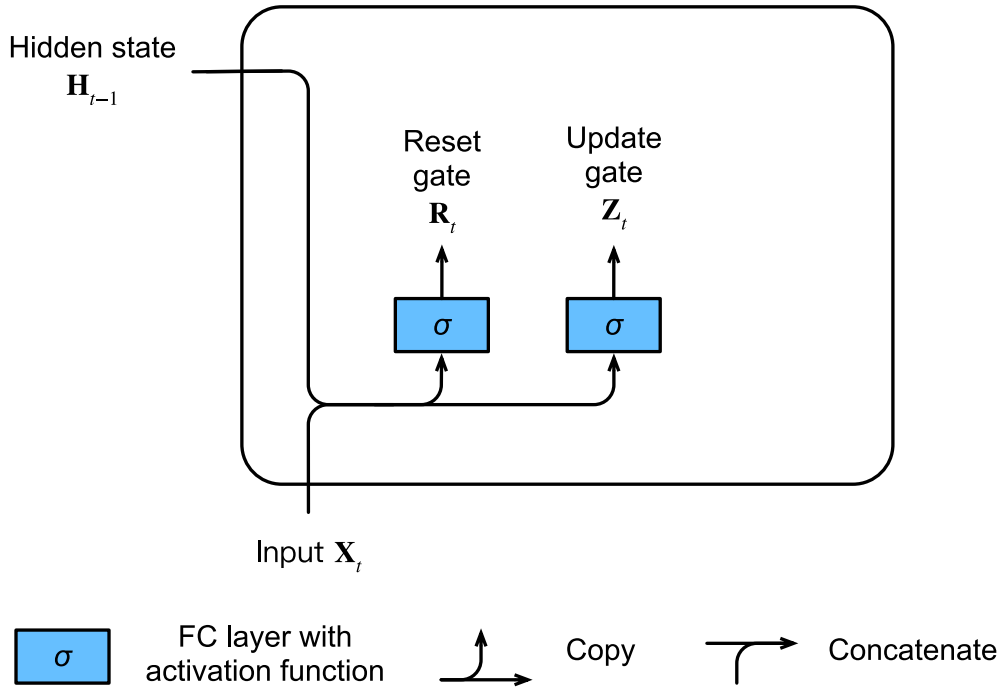
$\boldsymbol{Z}_t$: Binary vector

$\boldsymbol{P}_t$: Binary vector

What function can we achieve?

Any problem here?

Fourth attempt



Memorize, Skip, Reset

How much we use $\boldsymbol{X}_t$ and $\boldsymbol{H}_{t-1}$ to get $\boldsymbol{H}_t$

$$\boldsymbol{H}_t = \phi(\boldsymbol{X}_t\boldsymbol{W}_{xh} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{hh} + \boldsymbol{b}_h)$$

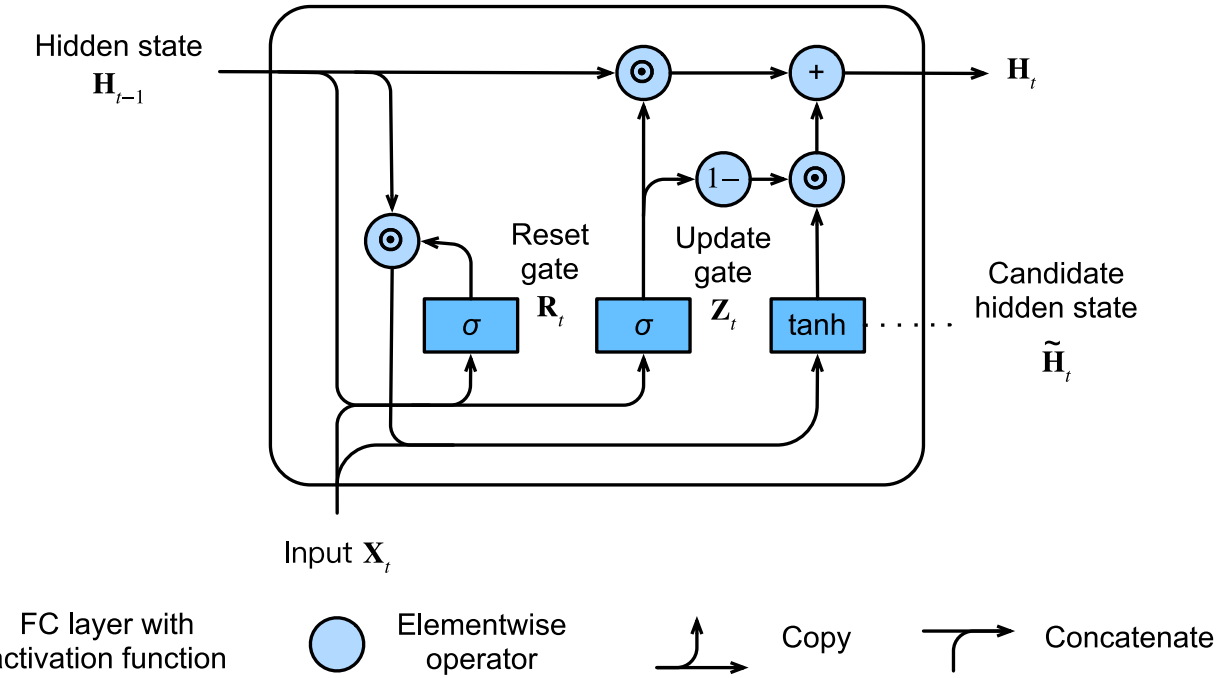
We need some gates to turn the information on or off

$$\boldsymbol{H}_t = (1 - \boldsymbol{Z}_t)\phi(\boldsymbol{X}_t\boldsymbol{W}_{xh} + (\boldsymbol{R}_t \odot \boldsymbol{H}_{t-1})\boldsymbol{W}_{hh} + \boldsymbol{b}_h) + \boldsymbol{Z}_t \odot \boldsymbol{H}_{t-1}$$

$\boldsymbol{R}_t$: Binary vector

$\boldsymbol{Z}_t$: Binary vector

Gated recurrent units (GRUs)



Memorize, Skip, Reset

How much we use $\boldsymbol{X}_t$ and $\boldsymbol{H}_{t-1}$ to get $\boldsymbol{H}_t$

$$\boldsymbol{H}_t = \phi(\boldsymbol{X}_t\boldsymbol{W}_{xh} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{hh} + \boldsymbol{b}_h)$$

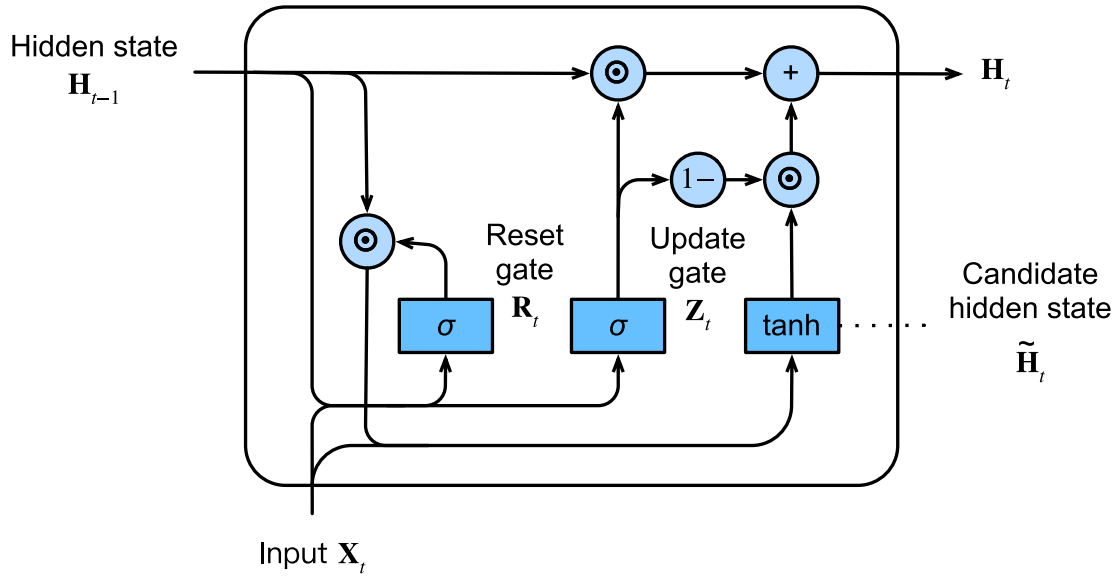
We need some gates to turn the information on or off

$$\boldsymbol{H}_t = (1 - \boldsymbol{Z}_t)\odot\tilde{\boldsymbol{H}}_t + \boldsymbol{Z}_t\odot\boldsymbol{H}_{t-1}$$
$$\tilde{\boldsymbol{H}}_t = tanh(\boldsymbol{X}_t\boldsymbol{W}_{xh} + (\boldsymbol{R}_t\odot\boldsymbol{H}_{t-1})\boldsymbol{W}_{hh} + \boldsymbol{b}_h)$$
$$\boldsymbol{R}_t = \delta(\boldsymbol{X}_t\boldsymbol{W}_{xr} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{hr} + \boldsymbol{b}_r)$$
$$\boldsymbol{Z}_t = \delta(\boldsymbol{X}_t\boldsymbol{W}_{xz} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{hz} + \boldsymbol{b}_z)$$

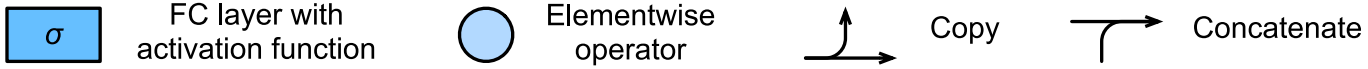
Analysis

$\mathbf{R}_t$, $\mathbf{Z}_t$: 0, 1

Gated recurrent units (GRUs)



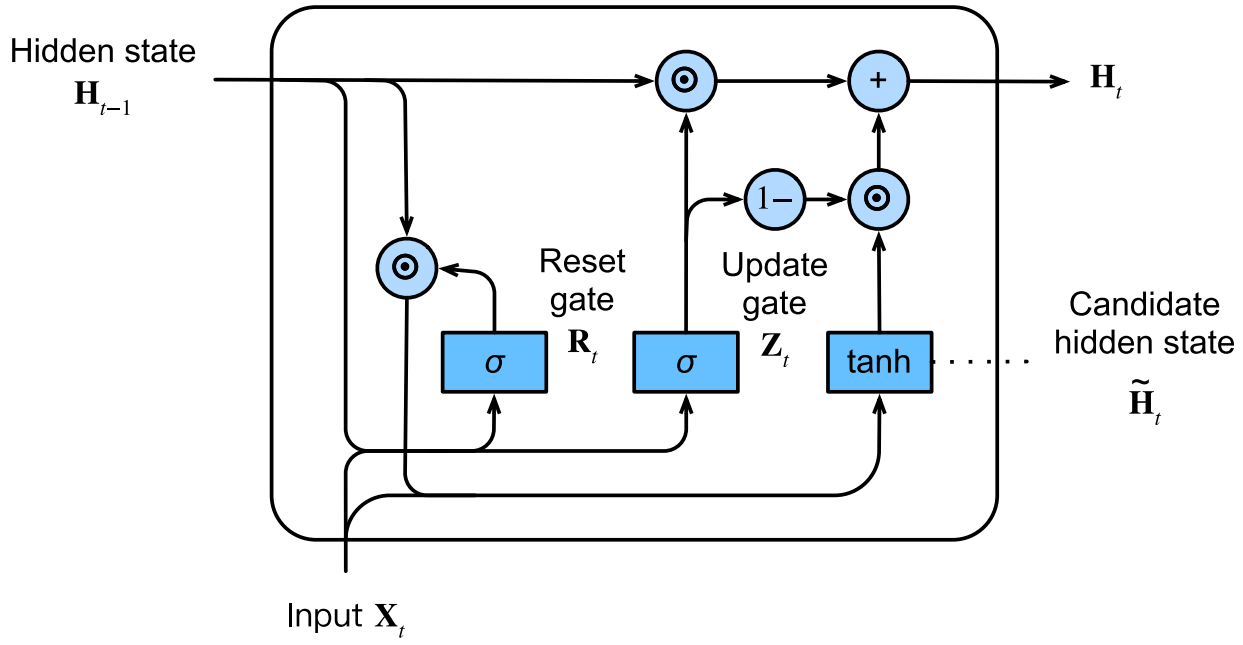
$$\boldsymbol{H}_t = (1 - \boldsymbol{Z}_t) \odot \tilde{\boldsymbol{H}}_t + \boldsymbol{Z}_t \odot \boldsymbol{H}_{t-1}$$
$$\tilde{\boldsymbol{H}}_t = tanh(\boldsymbol{X}_t \boldsymbol{W}_{xh} + (\boldsymbol{R}_t \odot \boldsymbol{H}_{t-1}) \boldsymbol{W}_{hh} + \boldsymbol{b}_h)$$
$$\boldsymbol{R}_t = \delta(\boldsymbol{X}_t \boldsymbol{W}_{xr} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hr} + \boldsymbol{b}_r)$$
$$\boldsymbol{Z}_t = \delta(\boldsymbol{X}_t \boldsymbol{W}_{xz} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hz} + \boldsymbol{b}_z)$$



Reset gates help capture short-term dependencies in sequences

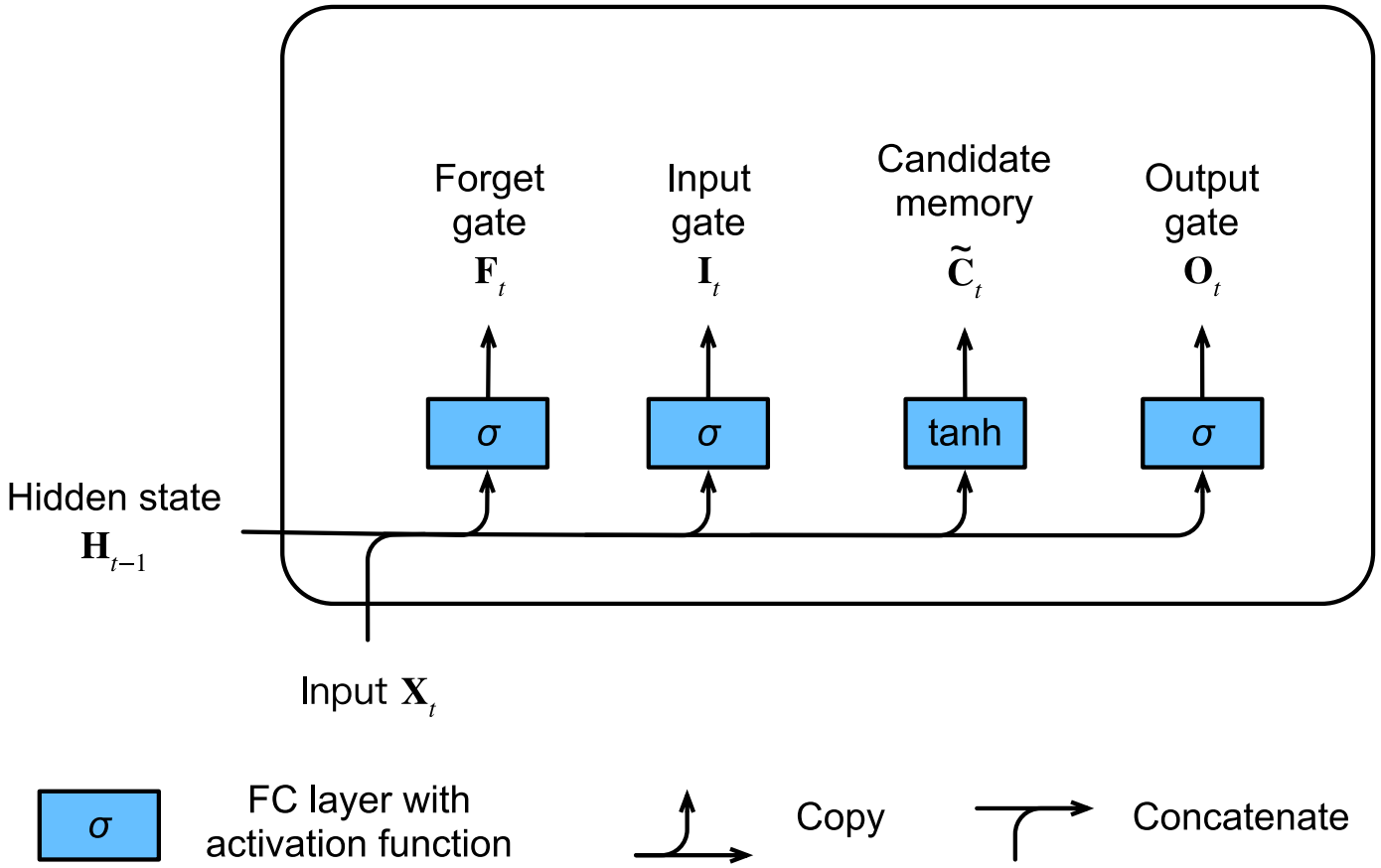
Update gates help capture long-term dependencies in sequences

Other idea



Memory is important
Could we have a dedicated memory cell?

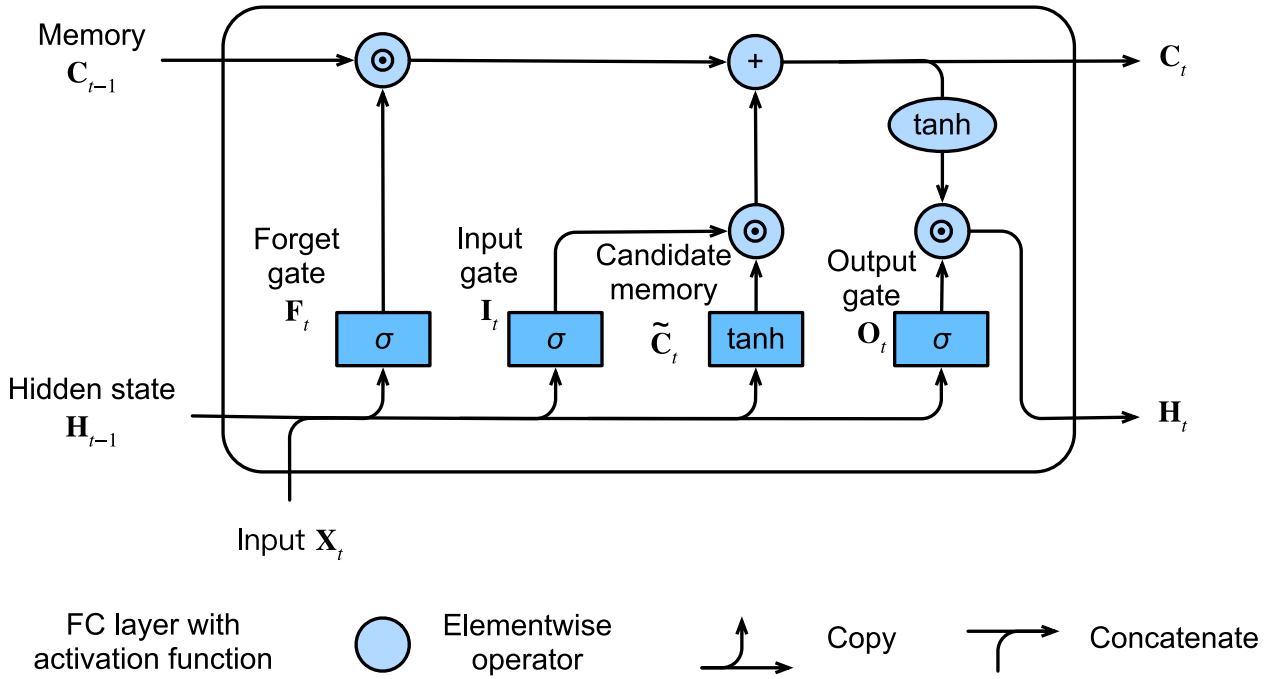
An alternative



Why do we need those gates?

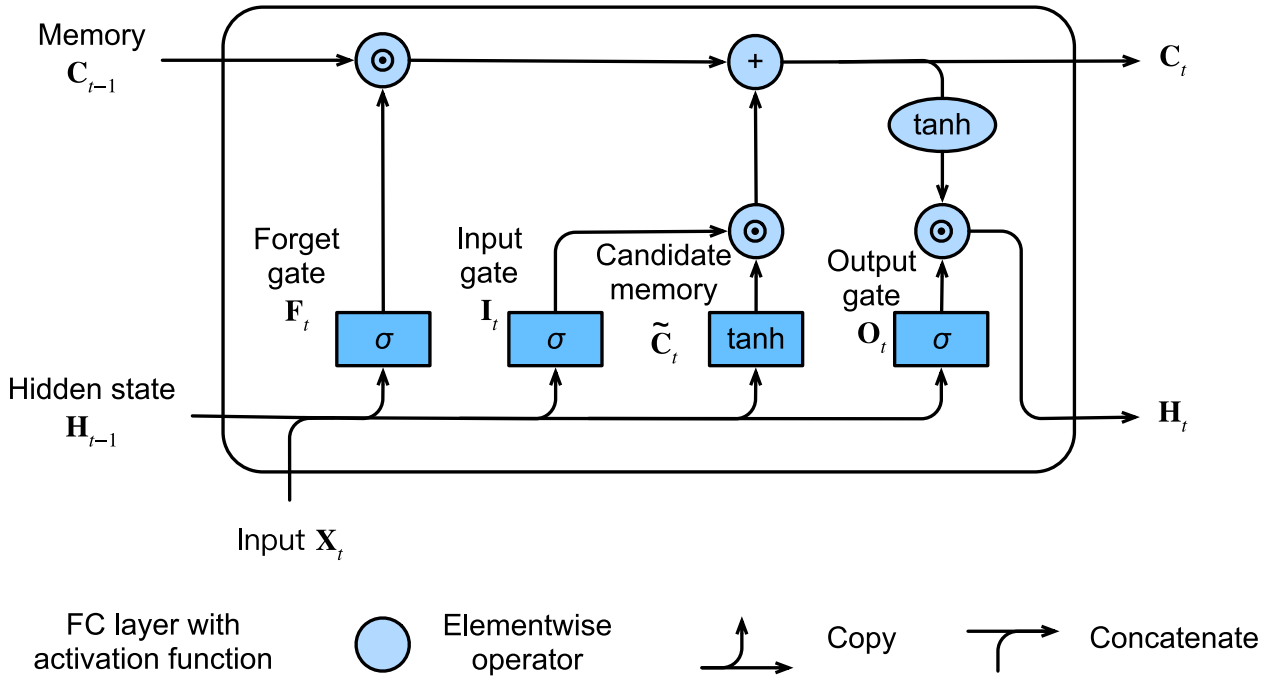
Any idea how we can combine those gate together based on their name?

Long short-term memory (LSTM)



$$\boldsymbol{H}_t = \boldsymbol{O}_t \odot tanh(\boldsymbol{C}_t)$$
$$\boldsymbol{C}_t = \boldsymbol{F}_t \odot \boldsymbol{C}_{t-1} + \boldsymbol{I}_t \odot \tilde{\boldsymbol{C}}_t$$
$$\tilde{\boldsymbol{C}}_t = tanh(\boldsymbol{X}_t \boldsymbol{W}_{xc} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hc} + \boldsymbol{b}_c)$$
$$\boldsymbol{I}_t = \delta(\boldsymbol{X}_t \boldsymbol{W}_{xi} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hi} + \boldsymbol{b}_i)$$
$$\boldsymbol{F}_t = \delta\left(\boldsymbol{X}_t \boldsymbol{W}_{xf} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hf} + \boldsymbol{b}_f\right)$$
$$\boldsymbol{O}_t = \delta(\boldsymbol{X}_t \boldsymbol{W}_{xo} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{ho} + \boldsymbol{b}_o)$$

Long short-term memory (LSTM)



The memory state is very similar to the candidate hidden state in GRU

It can alleviate vanishing and exploding gradients

To make you awake

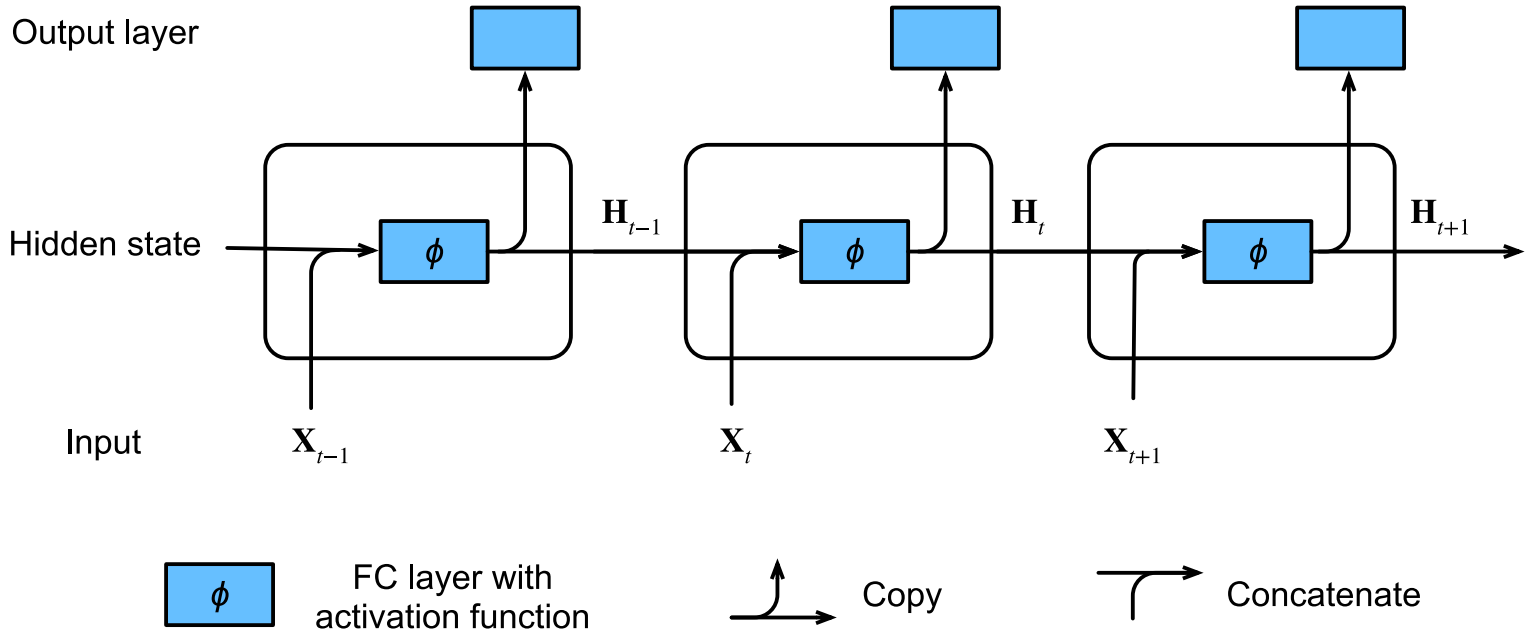
https://ureply.mobi/teacher

Today's agenda



- RNN++
  - GRU
  - LSTM
- Other models
  - Deep RNNs
  - Bidirectional model
  - Sequence to sequence model
    - Machine translation
- Attention

Recurrent neural networks (RNNs)



$$\boldsymbol{H}_t = \phi(\boldsymbol{X}_t \boldsymbol{W}_{xh} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hh} + \boldsymbol{b}_h)$$
$$\boldsymbol{O}_t = \delta(\boldsymbol{H}_t \boldsymbol{W}_{ho} + \boldsymbol{b}_o)$$

Is the model too simple?

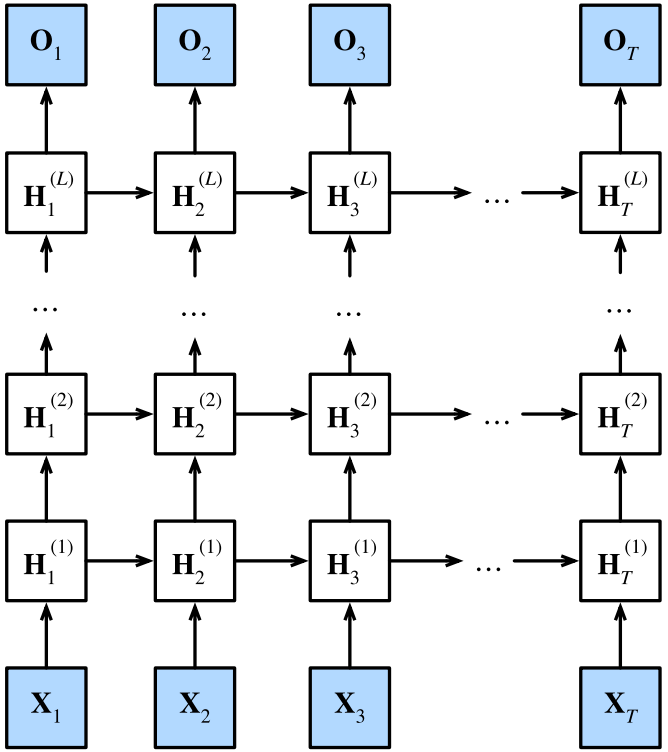
How to increase the model capacity?

Deep RNN/LSTM/GRU

Hopefully,

High-level trend
(bear or bull market)

Shorter-term
temporal dynamics
(data of the day)



$$\boldsymbol{H}_t^{(l)} = \phi_l\left(\boldsymbol{H}_t^{(l-1)}\boldsymbol{W}_{xh}^{(l)} + \boldsymbol{H}_{t-1}^{(l)}\boldsymbol{W}_{hh}^{(l)} + \boldsymbol{b}_h^{(l)}\right)$$

$$\boldsymbol{O}_t = \delta(\boldsymbol{H}_t^{(L)}\boldsymbol{W}_{ho} + \boldsymbol{b}_o)$$

Any further possible improvement?

Fill in the sentence



- I am ___ hungry

Fill in the sentence



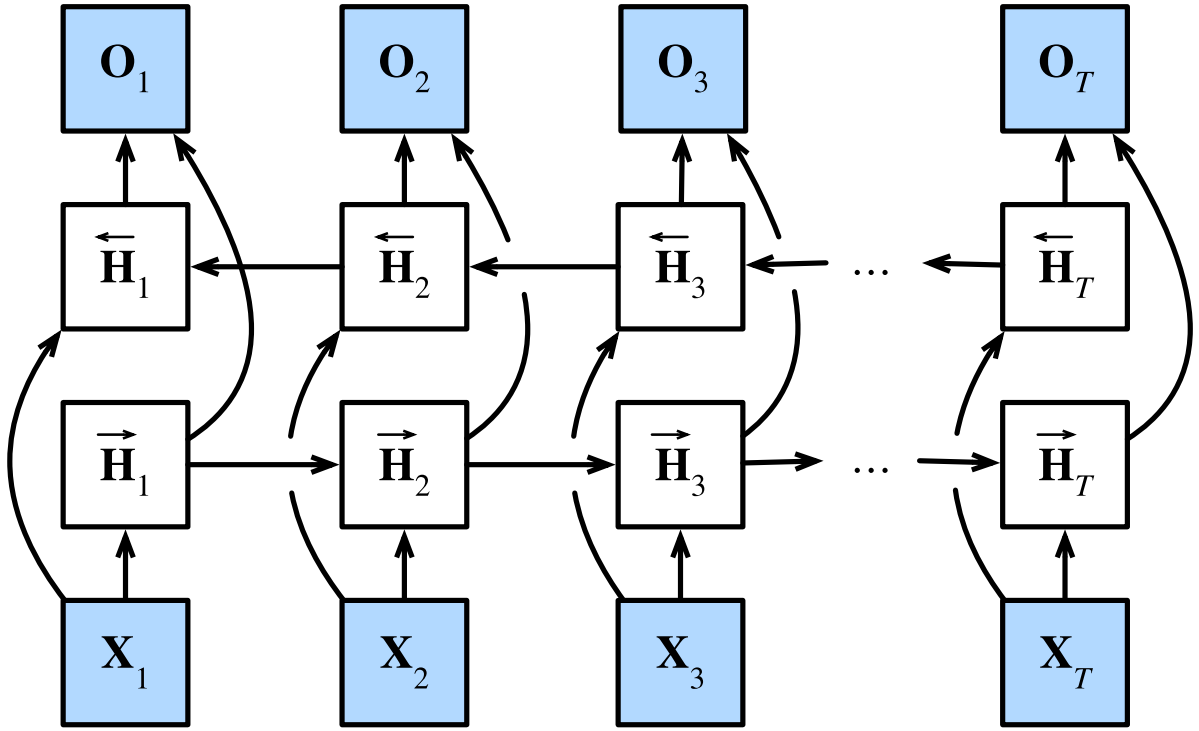
❖I am ___ hungry

The downstream information can be helpful

❖I am ___ hungry, and I can eat half a pig

❖I am ___ hungry, because I have already eaten half a pig

Bidirectional model



$$\overrightarrow{\boldsymbol{H}}_t = \phi\left(\boldsymbol{X}_t \boldsymbol{W}_{xh}^{(f)} + \overrightarrow{\boldsymbol{H}}_{t-1} \boldsymbol{W}_{hh}^{(f)} + \boldsymbol{b}_h^{(f)}\right)$$

$$\overleftarrow{\boldsymbol{H}}_t = \phi\left(\boldsymbol{X}_t \boldsymbol{W}_{xh}^{(b)} + \overleftarrow{\boldsymbol{H}}_{t+1} \boldsymbol{W}_{hh}^{(b)} + \boldsymbol{b}_h^{(b)}\right)$$

$$\boldsymbol{O}_t = \delta(cat(\overrightarrow{\boldsymbol{H}}_t, \overleftarrow{\boldsymbol{H}}_t)\boldsymbol{W}_{ho} + \boldsymbol{b}_o)$$

Any limitations of bidirectional models?

Bidirectional model



- Limited applications
  - Can't be used to do forward prediction
  - Training with information of both direction, testing only one direction. Poor performance
  - Filling in missing words
  - Annotating tokens: name entity recognition
  - Sequence encoding: machine translation

- Slow
  - Very, very slow
  - Forward propagation requires both forward and backward recursions

Machine translation



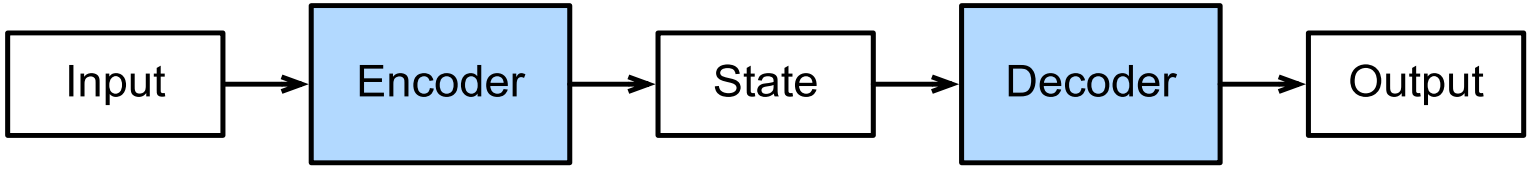
- Automatic translation of a sequence from one language to another
- Usually use word-level tokenization
- Example
  - Thank you
  - 谢谢你
  - شكرا
  - ありがとうございます
- Any observation?

Machine translation

- Sequences of different lengths
  - Not direct mapping

- Encoder: transform the input into a state with a fixed shape

- Decoder: map the encoded state to a variable-length sequence



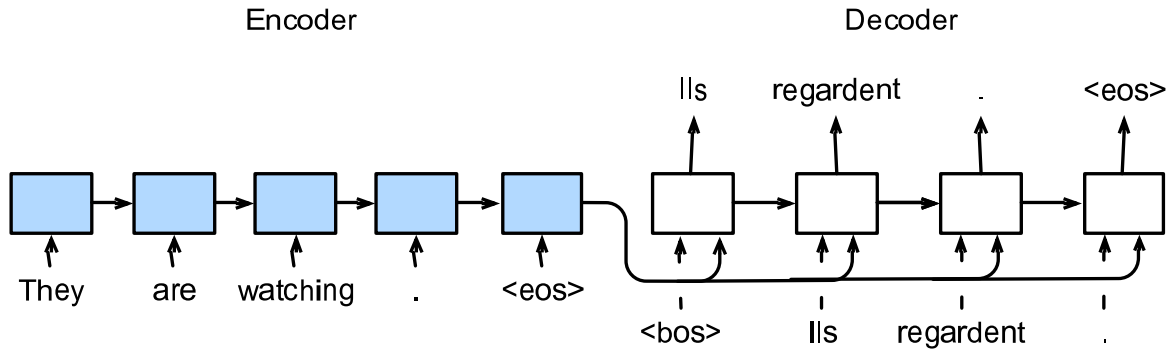
Sequence to sequence model

Encoder

$$\boldsymbol{h}_t = f(\boldsymbol{x}_t, \boldsymbol{h}_{t-1})$$
$$\boldsymbol{c} = q(\boldsymbol{h}_1, \ldots, \boldsymbol{h}_T)$$

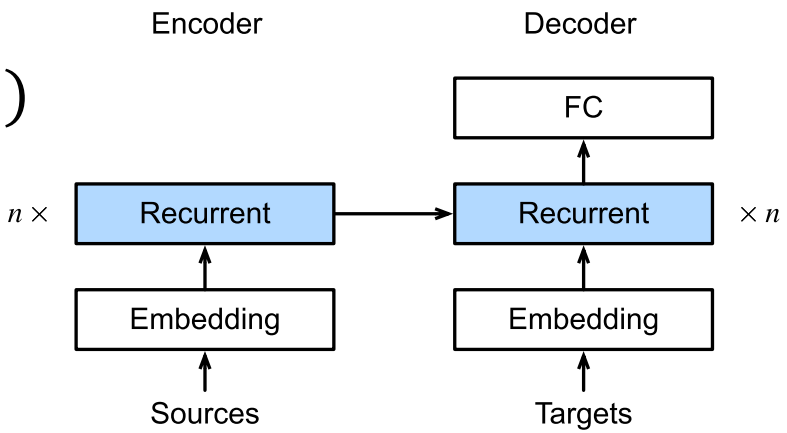
For example,

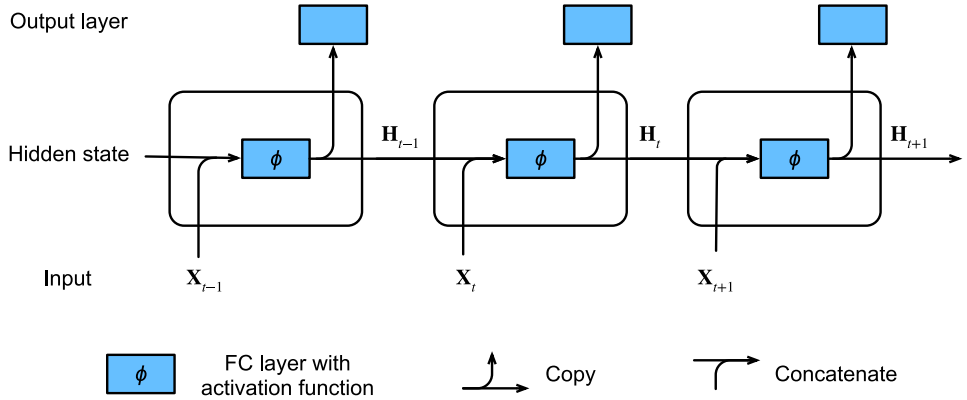
$$\boldsymbol{c} = q(\boldsymbol{h}_1, \ldots, \boldsymbol{h}_T) = \boldsymbol{h}_T$$



Decoder

$$\boldsymbol{s}_{t'} = g(y_{t'}, \boldsymbol{c}, \boldsymbol{s}_{t'-1})$$

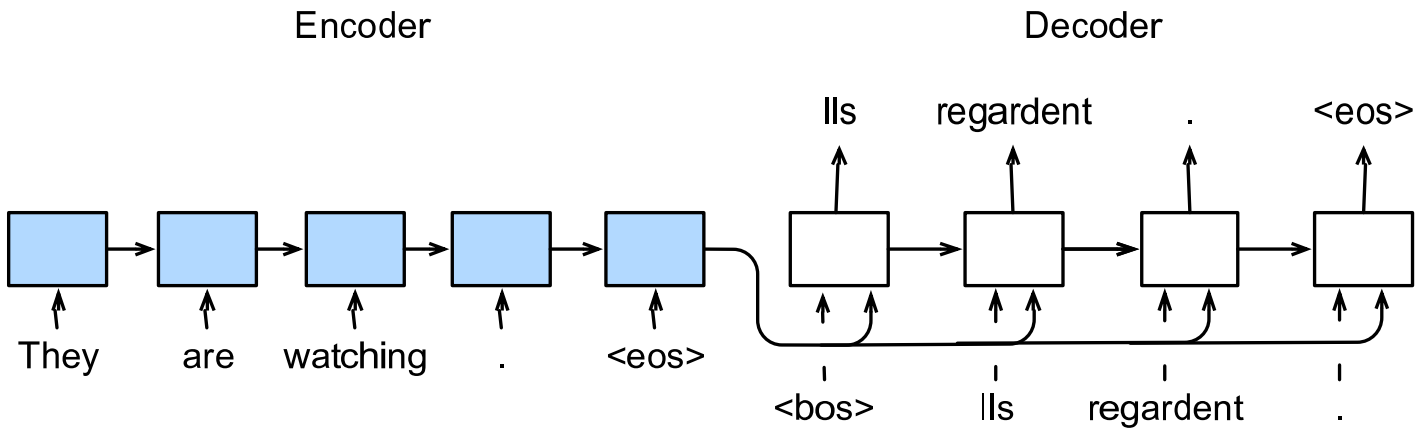




Sequence to sequence model



- Loss function
  - Each position: classification
  - Cross-entropy loss



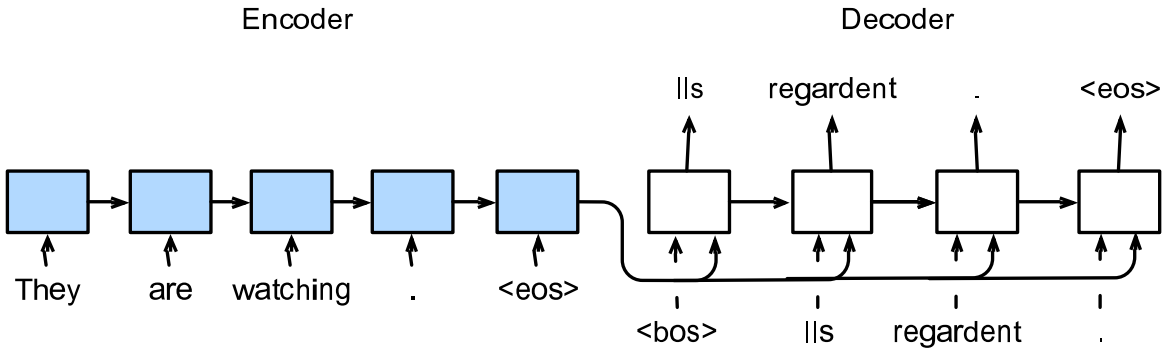
- Prediction
  - When predicting "<eos>", the prediction is complete

Sequence to sequence model



- Evaluation
  - Bilingual Evaluation Understudy (BLEU)

- p_n: precision of n-grams
  - $$\frac{Number\ of\ match\ n-grams}{Number\ of\ n-grams\ in\ predicted\ seq}$$



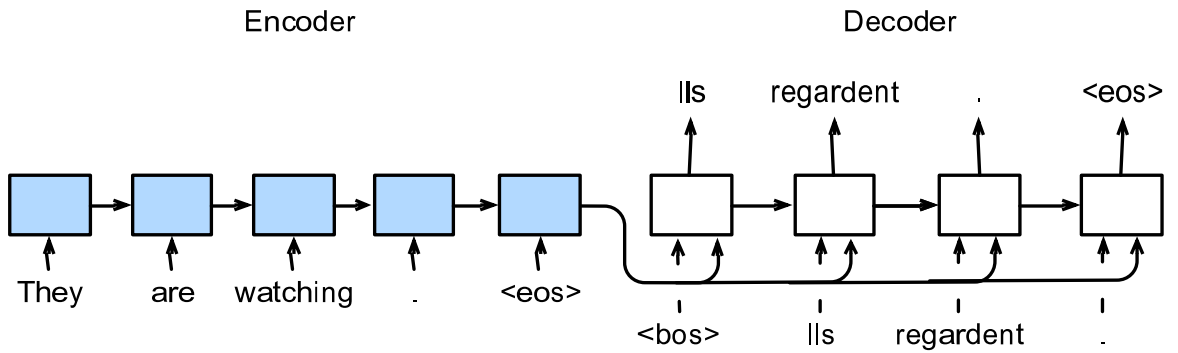
- Example
  - Target sequence: ABCDEF
  - Predicted sequence: ABBCD
  - $p_1 = \frac{4}{5}, p_2 = \frac{3}{4}, p_3 = \frac{1}{3}, p_4 = 0$

$$\text{BLEU} = \exp\left(\min\left(0, 1 - \frac{len_{target}}{len_{pred}}\right)\right) \prod_{n=1}^{k} p_n^{\frac{1}{2^n}}$$

The higher, the better

Sequence to sequence model

- We are using greedy search
  - The future results depend on current choice



- Problem of greedy search

Time step	1	2	3	4
A	0.5	0.1	0.2	0.0
B	0.2	0.4	0.2	0.2
C	0.2	0.3	0.4	0.2
<eos>	0.1	0.2	0.2	0.6

0.5*0.4*0.4*0.6=0.048

Time step	1	2	3	4
A	0.5	0.1	0.1	0.1
B	0.2	0.4	0.6	0.2
C	0.2	0.3	0.2	0.1
<eos>	0.1	0.2	0.1	0.6

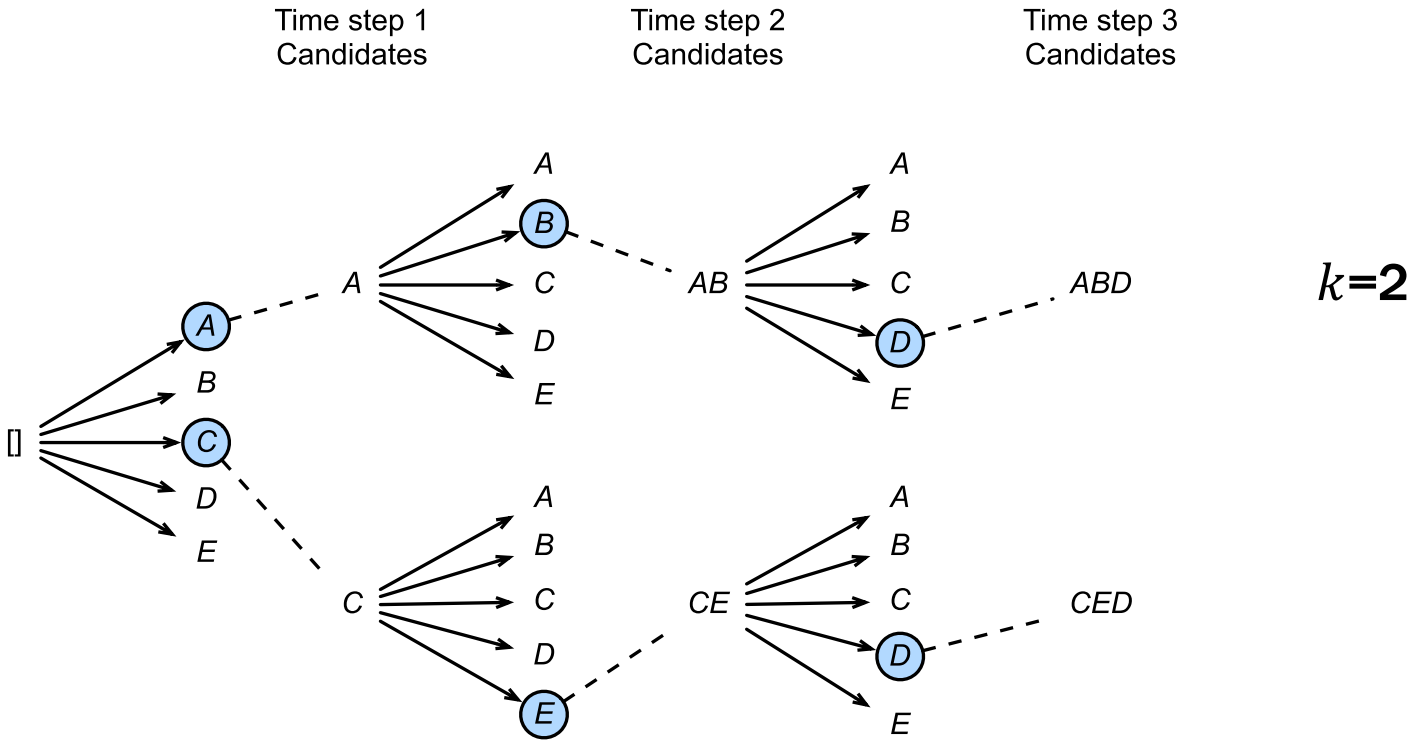
0.5*0.3*0.6*0.6=0.054

- Exhaustive search
  - Number of candidate tokens: 10000
  - Steps: 10

Computational cost?

Beam search

- An improved version of greedy search
- k: beam size
- Select k candidates from $k|D|$ possible choices each time



To make you awake

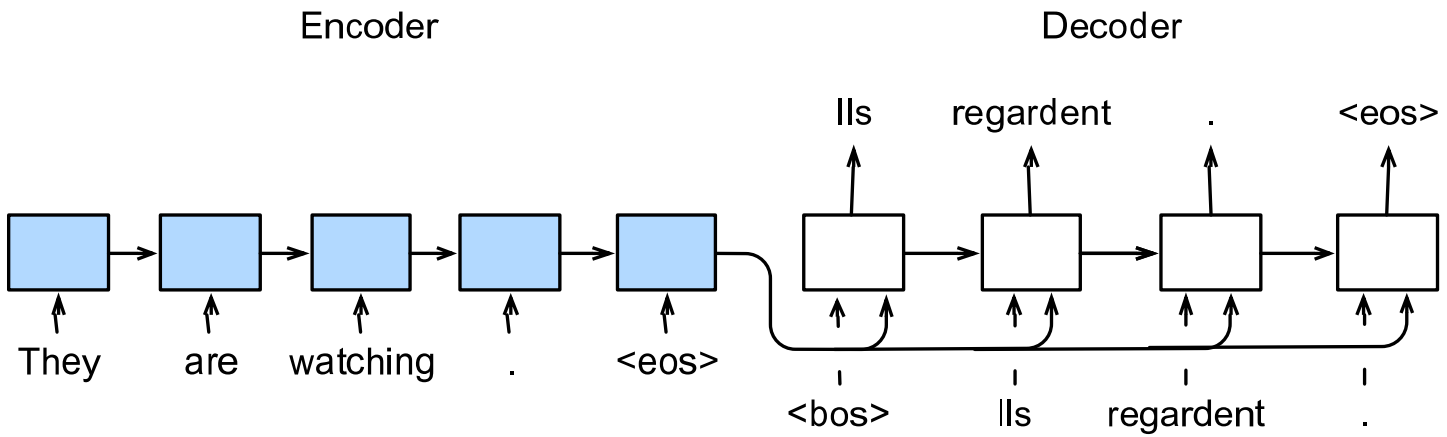
https://ureply.mobi/teacher

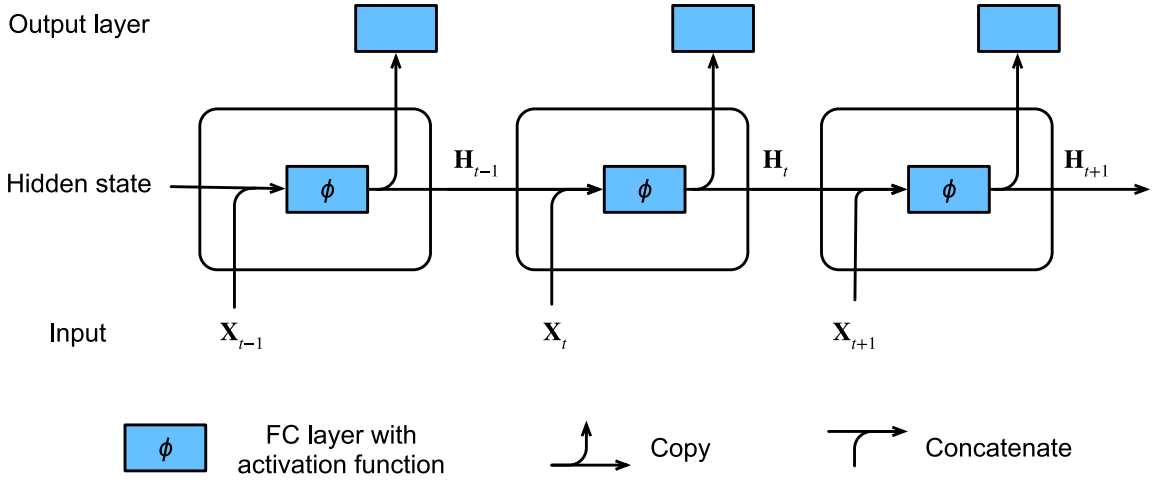
Today's agenda



- RNN++
  - GRU
  - LSTM
- Other models
  - Deep RNNs
  - Bidirectional model
  - Sequence to sequence model
    - Machine translation
- Attention

Machine translation





Hidden state is very important

We only have one hidden state to aggregate the information

The hidden state flows across the sentence

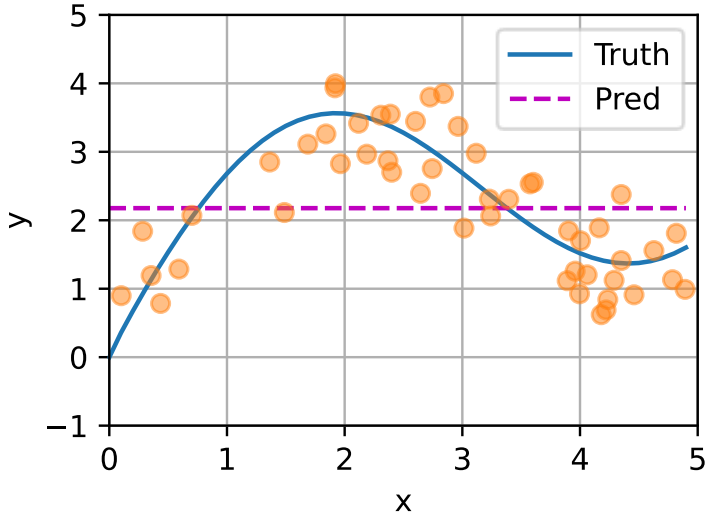
Can't ensure the representation for each word is proper

Any idea of encoding each word?

Representation using neighbor information

❖ Average

➢ $f(x) = \frac{1}{n}\sum_{i=1}^{n} y_i$

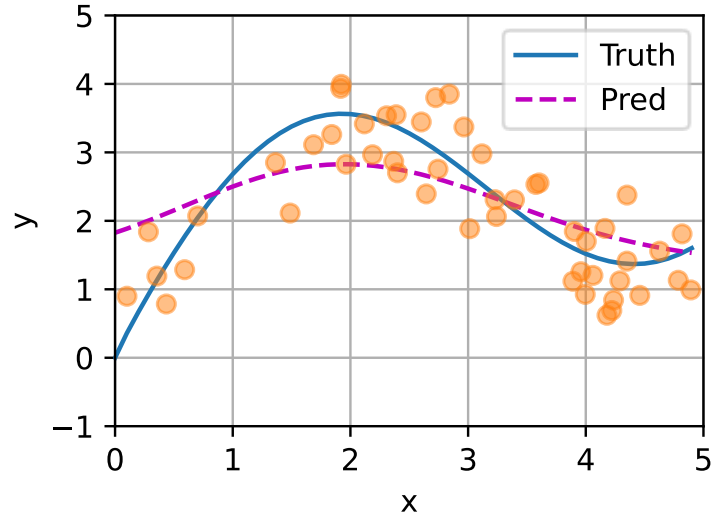


❖ Weighted average

➢ $f(x) = \sum_{i=1}^{n} \alpha(x, x_i) y_i$

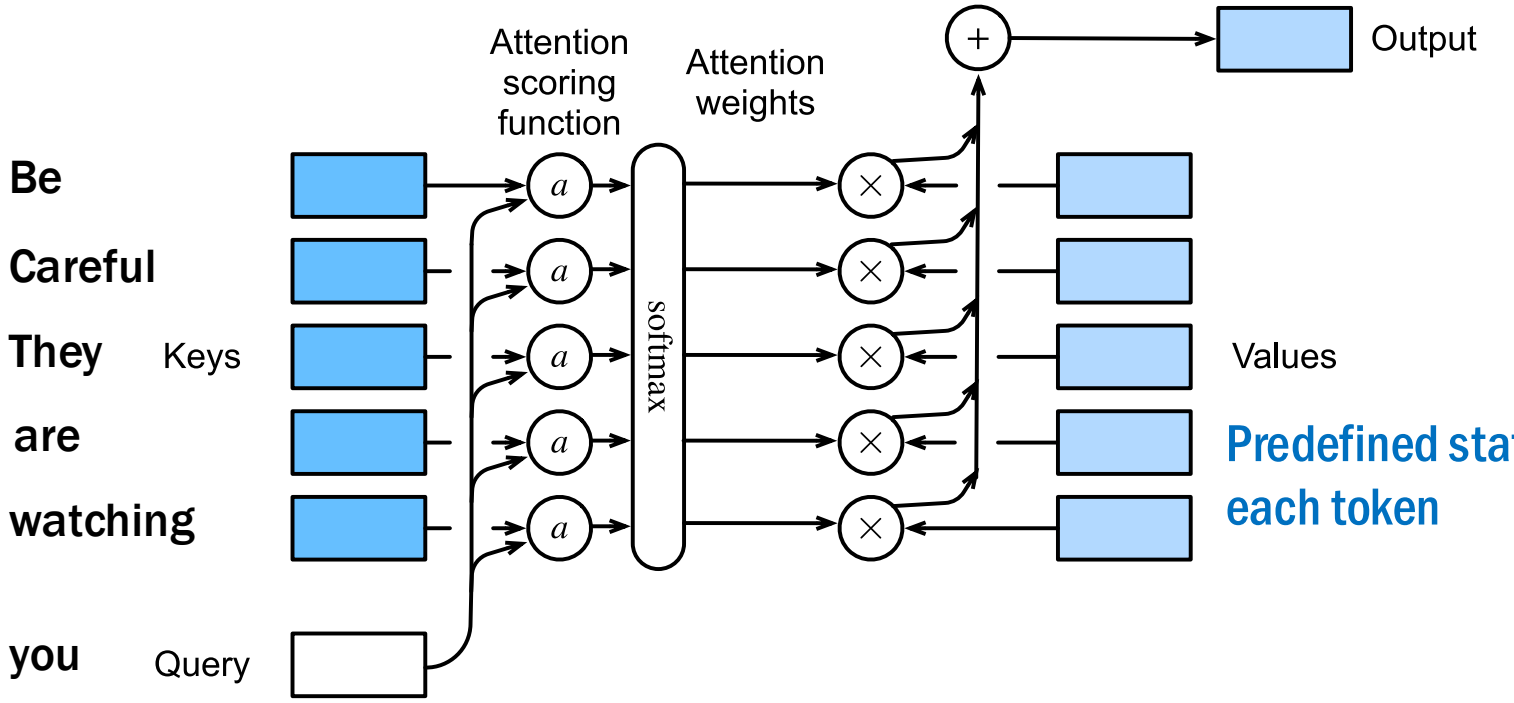
➢ $\alpha(x, x_i)$: related to similarity or distance

➢ Even more general: Attention!



Attention mechanism: Machine translation

Hidden state of each word: considering interactions with all the other words directly



Hidden state of Query

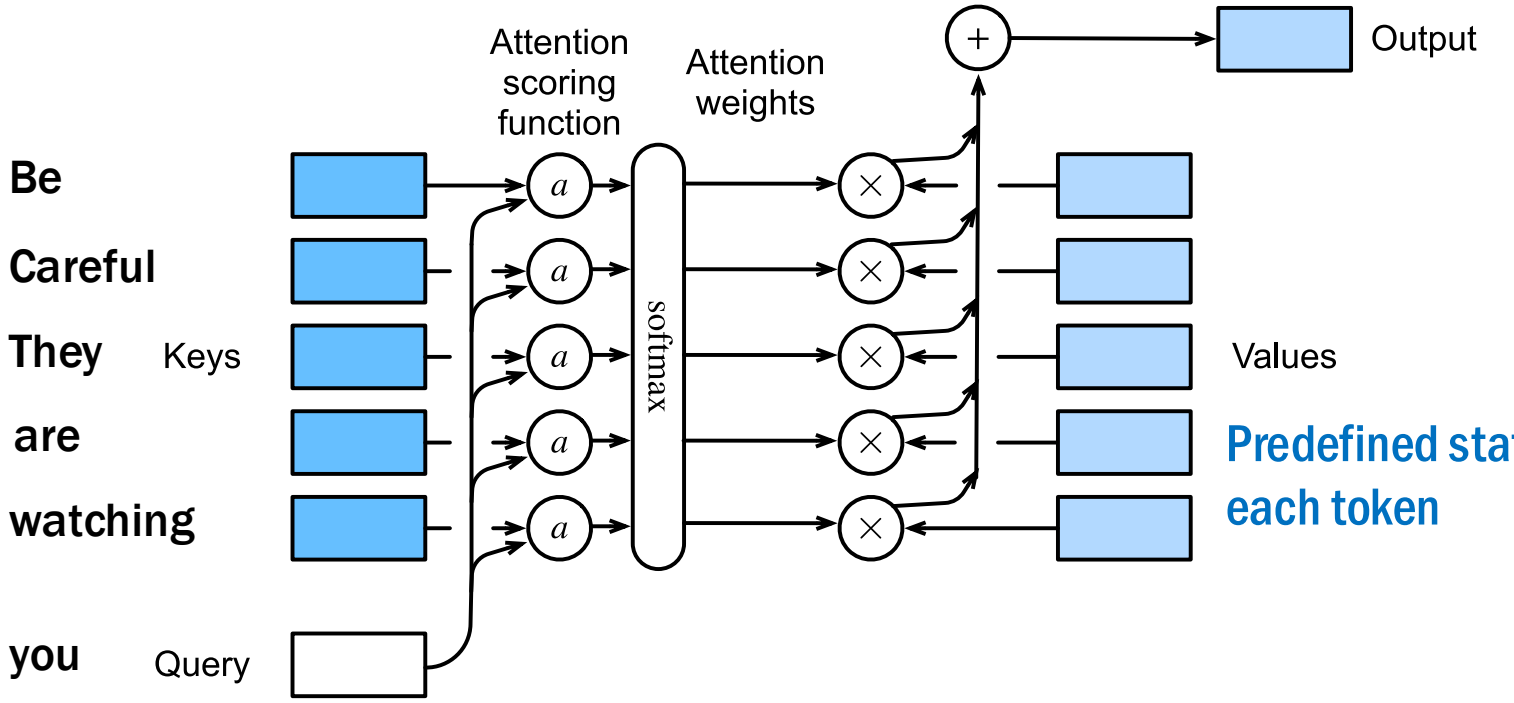
Based on the hidden state, we can do the downstream task

Predefined state for each token

$$Query\ hidden\ state\ =\ f(x)\ =\ \sum_{i=1}^{n} \alpha(x, x_i) y_i$$

Attention mechanism: Machine translation

Hidden state of each word: considering interactions with all the other words directly



Hidden state of Query

Based on the hidden state, we can do the downstream task

Predefined state for each token

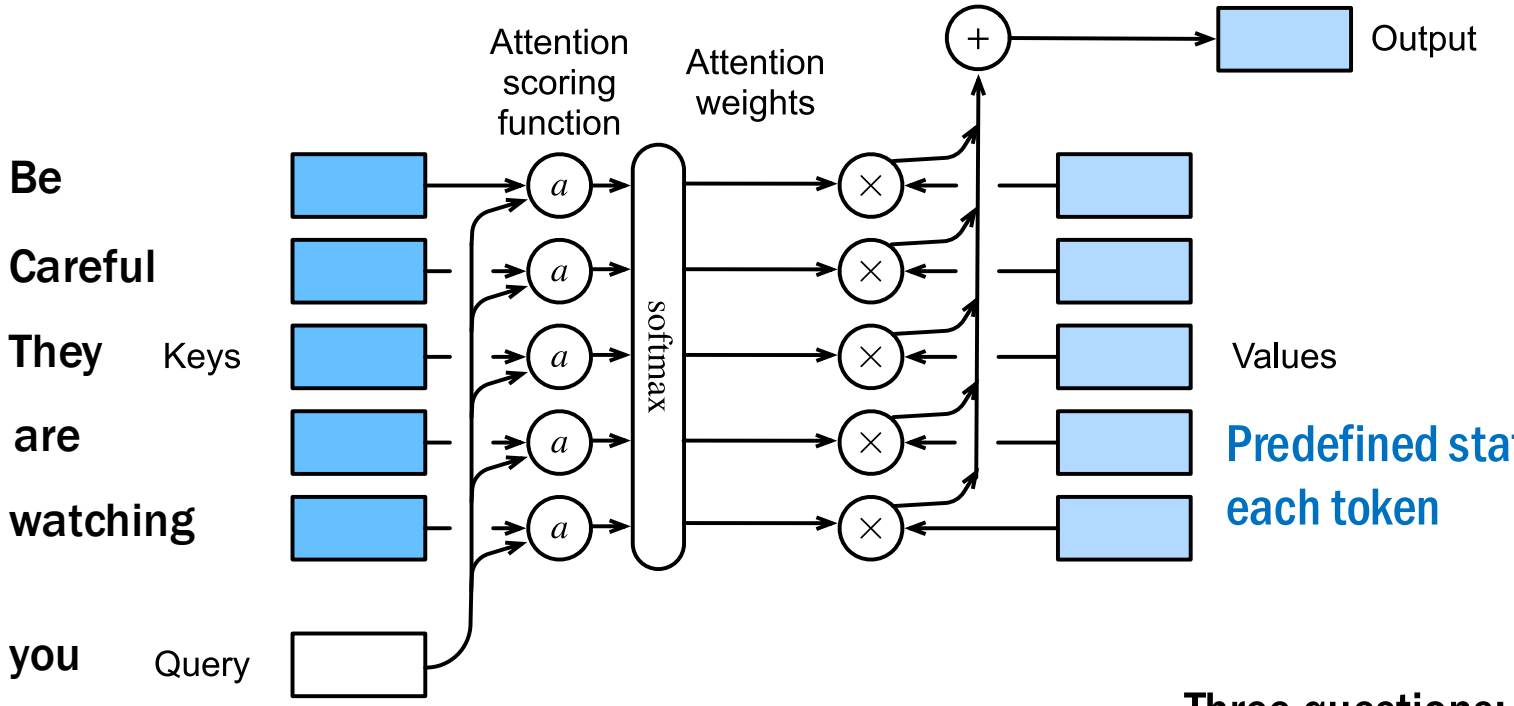
$$Query\ hidden\ state\ =\ f(x)\ =\ \sum_{i=1}^{n} \alpha(x, x_i) y_i$$

What is the difference between output hidden state and predefined state?

Context and neighbor information

Attention mechanism: Machine translation

Hidden state of each word: considering interactions with all the other words directly



Hidden state of Query

Based on the hidden state, we can do the downstream task

Predefined state for each token

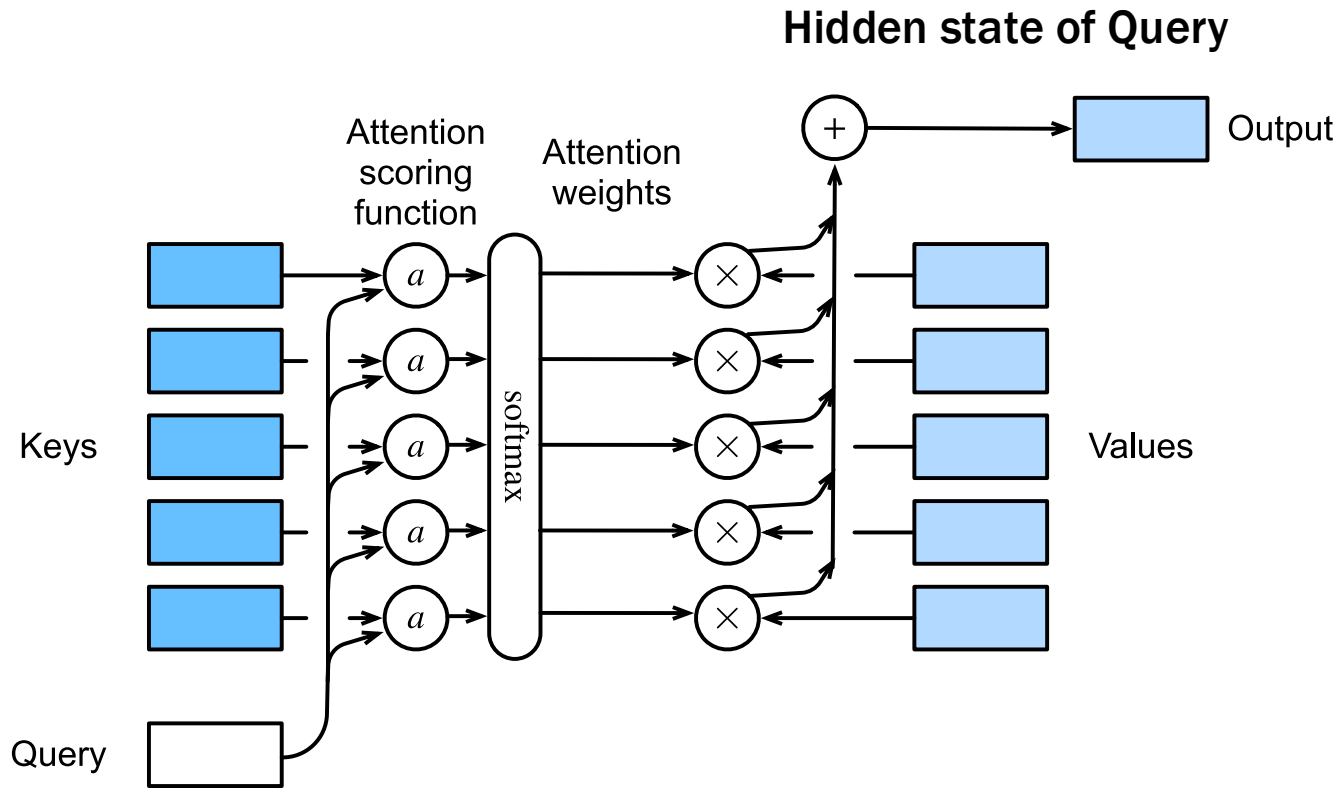
$$Query\ hidden\ state\ =\ f(x)\ =\ \sum_{i=1}^{n} \alpha(x, x_i) y_i$$

Three questions:

1. How to encode character?
2. How to define the predefined state?
3. How to define the attention weight function?

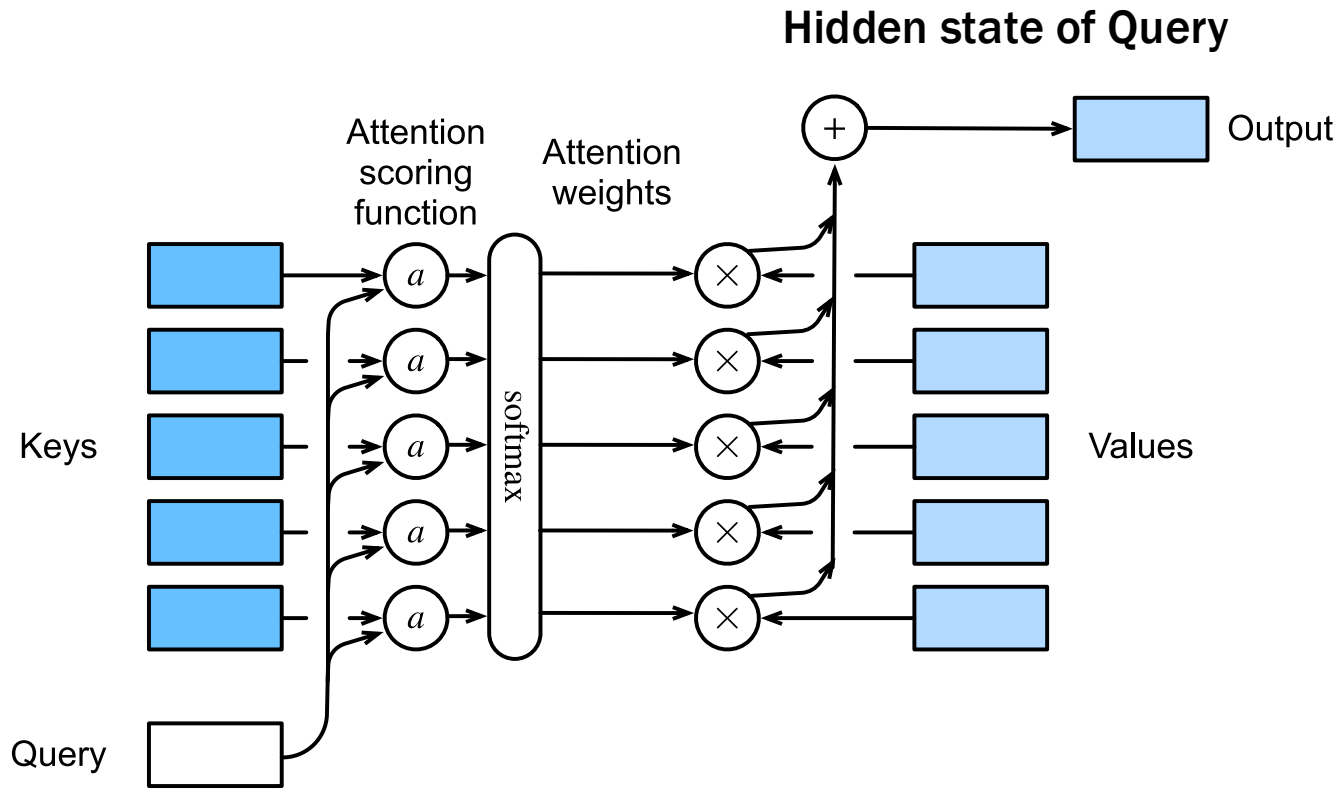
Attention mechanism: Machine translation

- Character encoding (keys)
  - One-hot encoding
  - Or FCNet (one-hot)
- Values
  - FCNet (one-hot)
- Attention weights (dot-product)
  - $\alpha(\boldsymbol{q}, \boldsymbol{k}_i) = \frac{\exp(a(\boldsymbol{q}, \boldsymbol{k}_i))}{\sum_{j=1}^{m} \exp(a(\boldsymbol{q}, \boldsymbol{k}_j))}$
  - $a(\boldsymbol{q}, \boldsymbol{k}) = \frac{\boldsymbol{q}^{\mathrm{T}} \boldsymbol{k}}{\sqrt{\mathrm{d}}}$



Attention mechanism: Machine translation

- Character encoding (keys)
  - One-hot encoding
  - Or FCNet (one-hot)
- Values
  - FCNet (one-hot)
- Attention weights (dot-product)
  - $\alpha(\boldsymbol{q}, \boldsymbol{k}_i) = \frac{\exp(a(\boldsymbol{q}, \boldsymbol{k}_i))}{\sum_{j=1}^{m} \exp(a(\boldsymbol{q}, \boldsymbol{k}_j))}$
  - $a(\boldsymbol{q}, \boldsymbol{k}) = \frac{\boldsymbol{q}^{\mathrm{T}} \boldsymbol{k}}{\sqrt{\mathrm{d}}}$

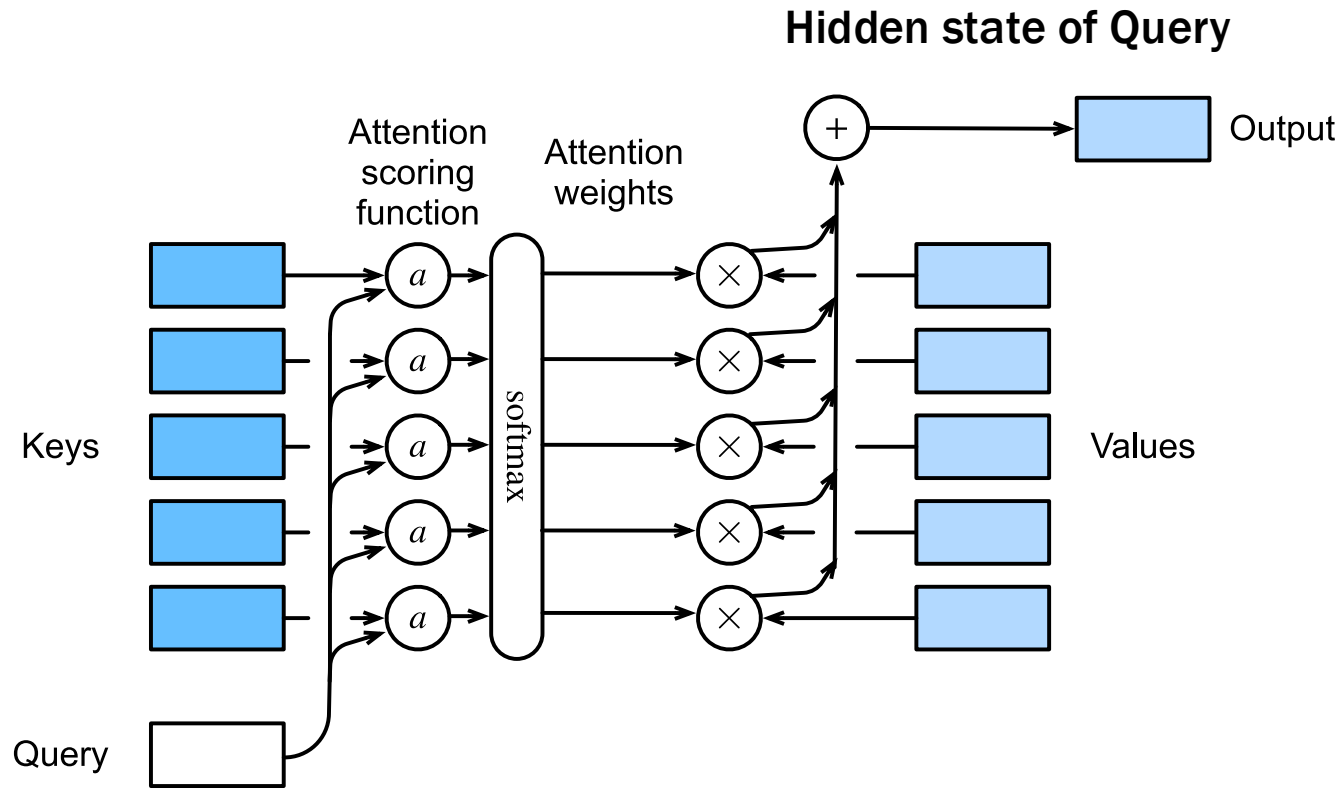


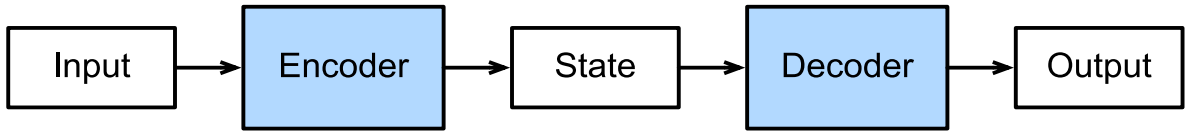
What are the learnable parameters in attention?

Attention mechanism: Machine translation



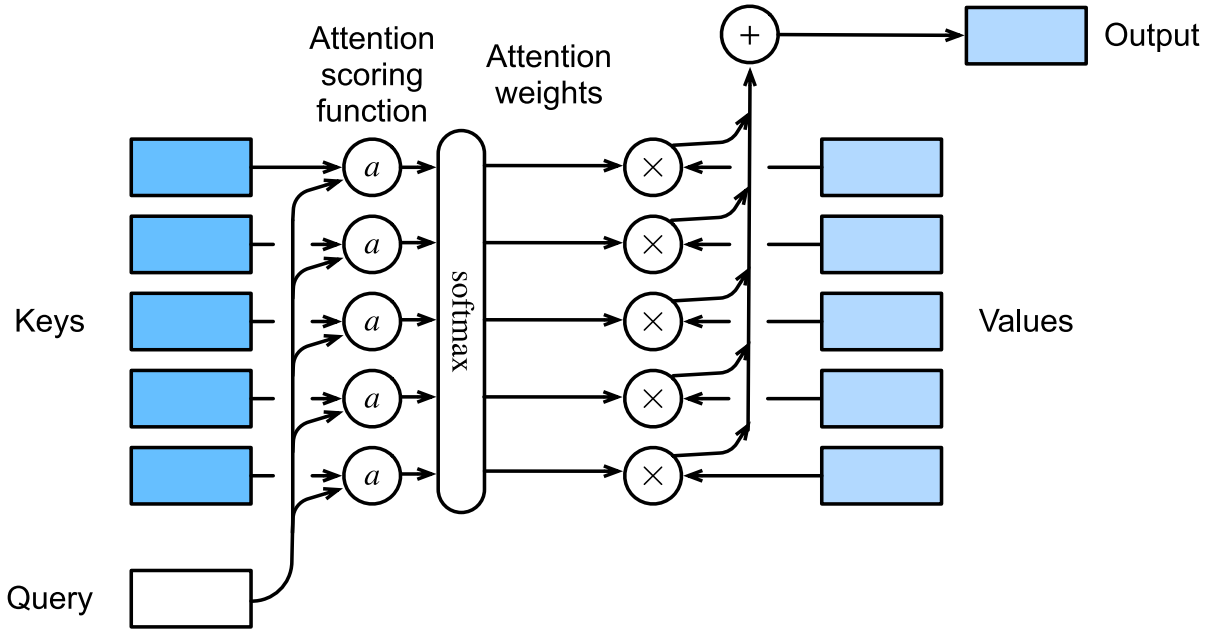
- Loss function
  - Cross-entropy for each word prediction
- Training
  - The common optimizers
  - Back-propagation





How to get the hidden state of all the tokens in the sentence?

Self-attention



For each input $\boldsymbol{x}_i$ in the sentence: $\boldsymbol{y}_i = \sum_{j=1}^{m} \alpha(\boldsymbol{x}_i, \boldsymbol{x}_j)\boldsymbol{v}_j$

$\boldsymbol{y}_i$: outputted hidden state for downstream classification or other tasks

Including the attention against the $\boldsymbol{x}_i$ itself

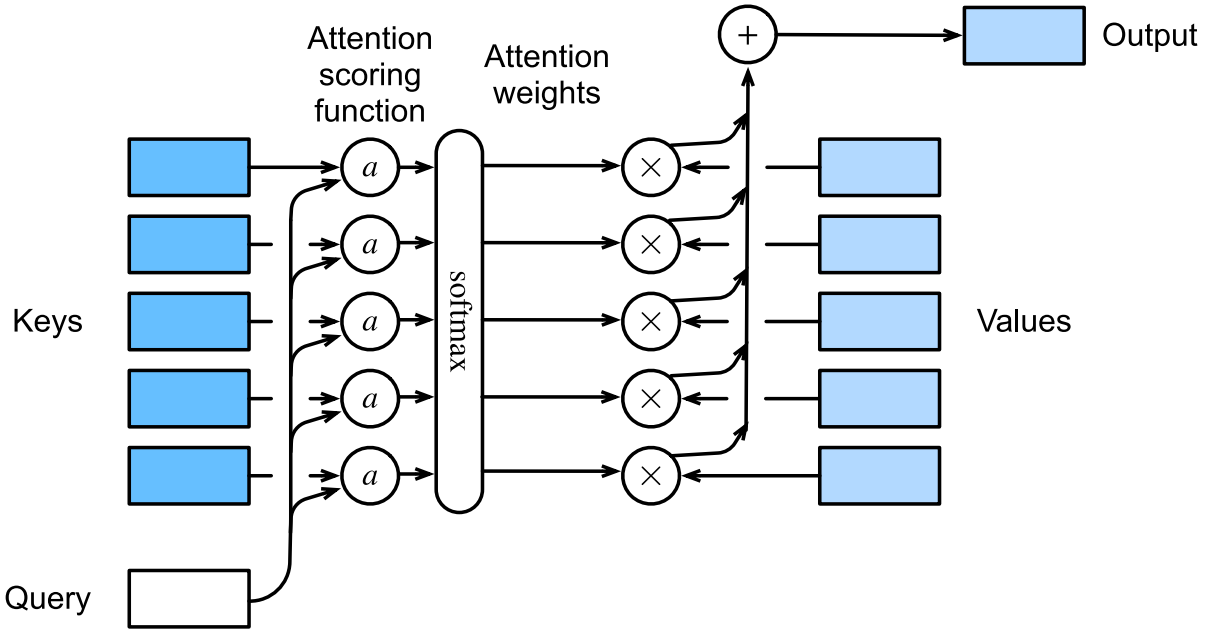
Can be computed in parallel

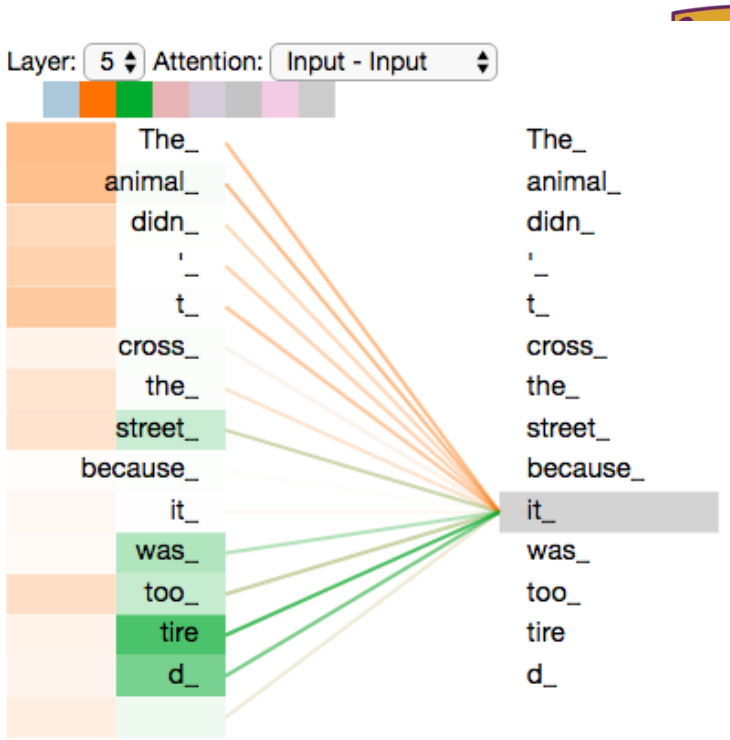
Get the encoding of each token by considering the context

Attention against the sentence itself

Encoding and embedding are the most important things for NLP

Self-attention





For each input x_i in the sentence: $y_i = \sum_{j=1}^{m} \alpha(x_i, x_j) v_j$

y_i: outputted hidden state for downstream classification or other tasks

Including the attention against the x_i itself

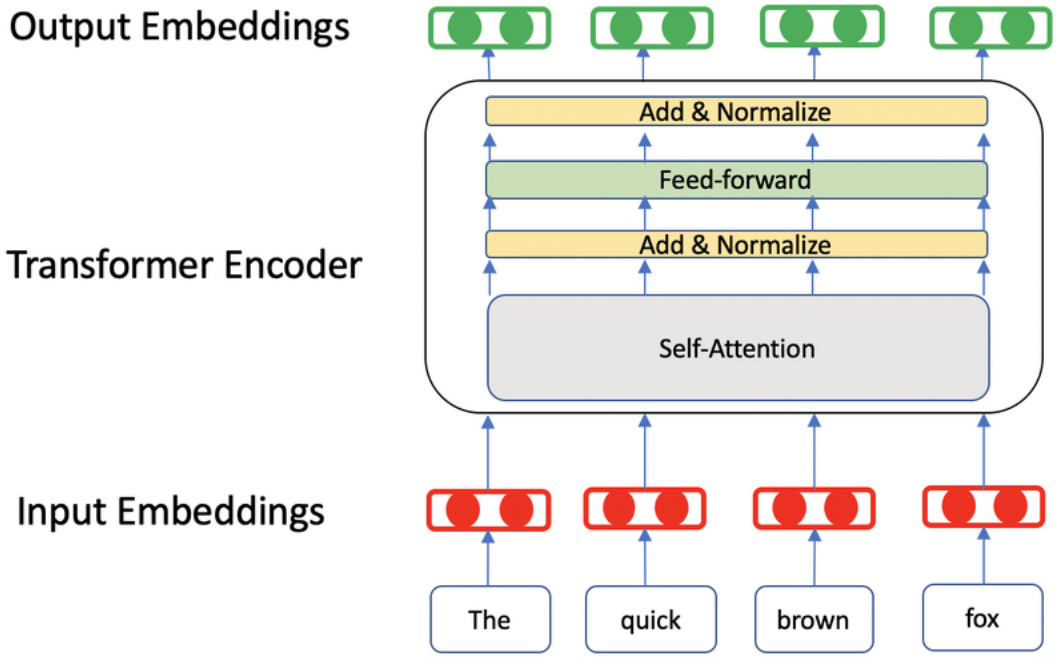
Can be computed in parallel

Get the encoding of each token by considering the context

Attention against the sentence itself

Encoding and embedding are the most important things for NLP

Self-attention



For each input $\boldsymbol{x}_i$ in the sentence: $\boldsymbol{y}_i = \sum_{j=1}^{m} \alpha(\boldsymbol{x}_i, \boldsymbol{x}_j)\boldsymbol{v}_j$

$\boldsymbol{y}_i$: outputted hidden state for downstream classification or other tasks

Including the attention against the $\boldsymbol{x}_i$ itself

Can be computed in parallel

Long-term dependency?



Time series

Sequences

Text

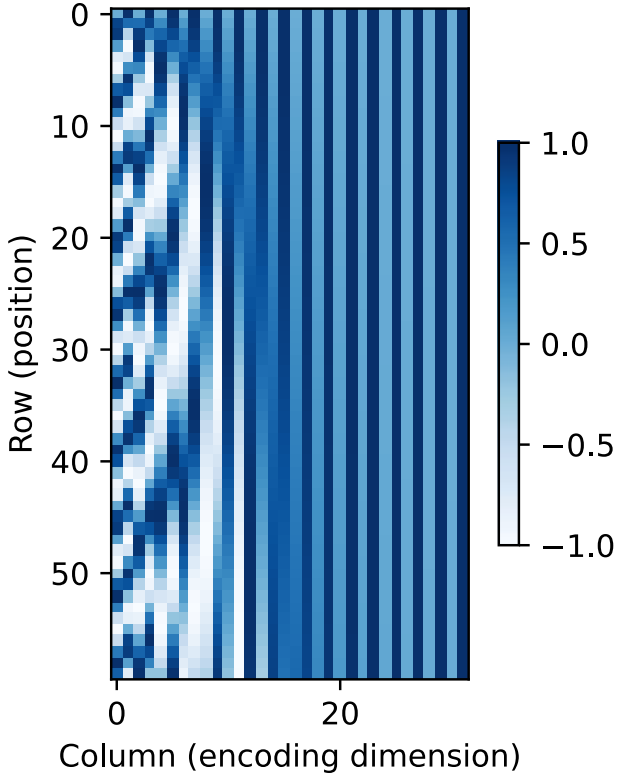
...

Order information

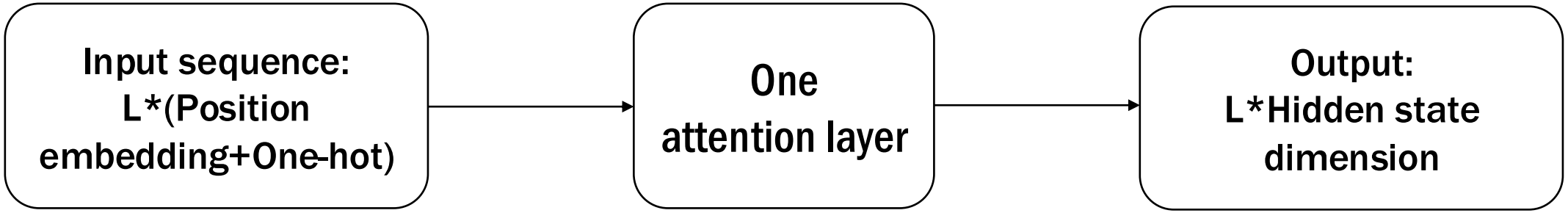
Position encoding



- $\boldsymbol{X} \in R^{n*d}$: d-dimensional embeddings for n tokens of a sequence
- $\mathbf{P} \in R^{n*d}$
  - The value of the ith row and the $(2j)^{th}$ or the $(2j+1)^{th}$ column
  - $p_{i,2j} = \sin(\frac{i}{10000^{\frac{2j}{d}}})$
  - $p_{i,2j+1} = \cos(\frac{i}{10000^{\frac{2j}{d}}})$
- Weird function but people use it
- Absolute & relative positional information
- Input dimension = Raw encoding dimension + PE dimension



Input and output of attention layer



Increase complexity?

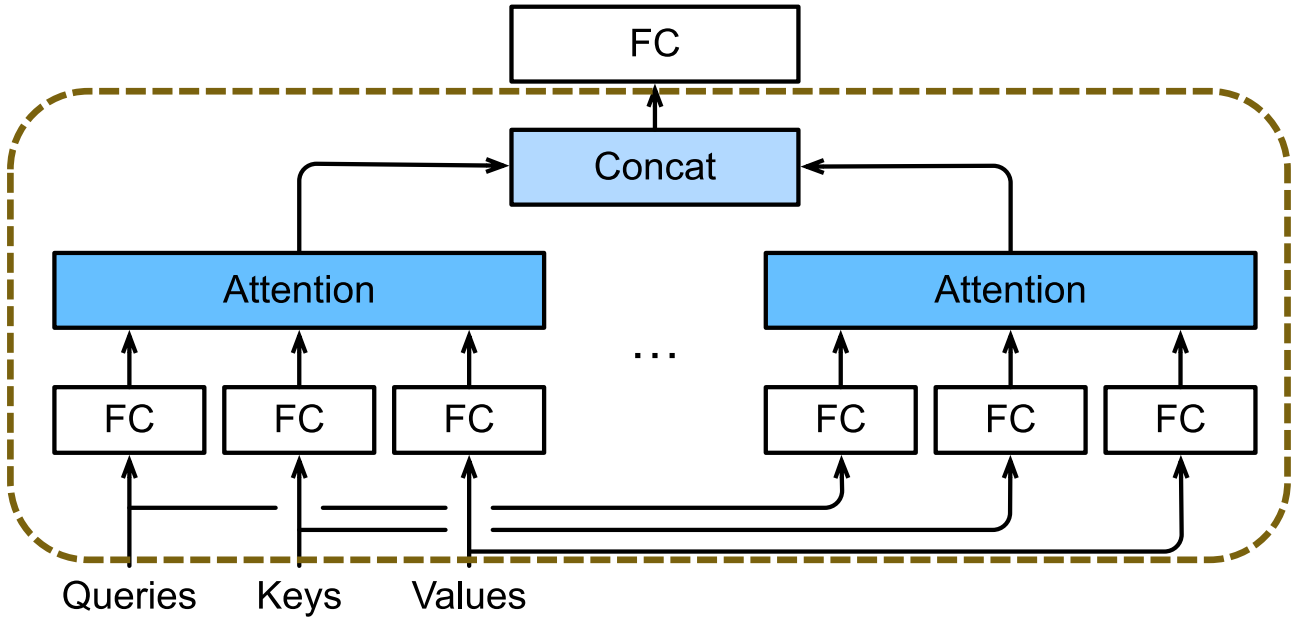
Multi-head attention and stack more attention layers

Multi-head attention

Why do we need FCNet here?

Where do we apply the FC? Token or sentence?

How is the FC parameter shared?

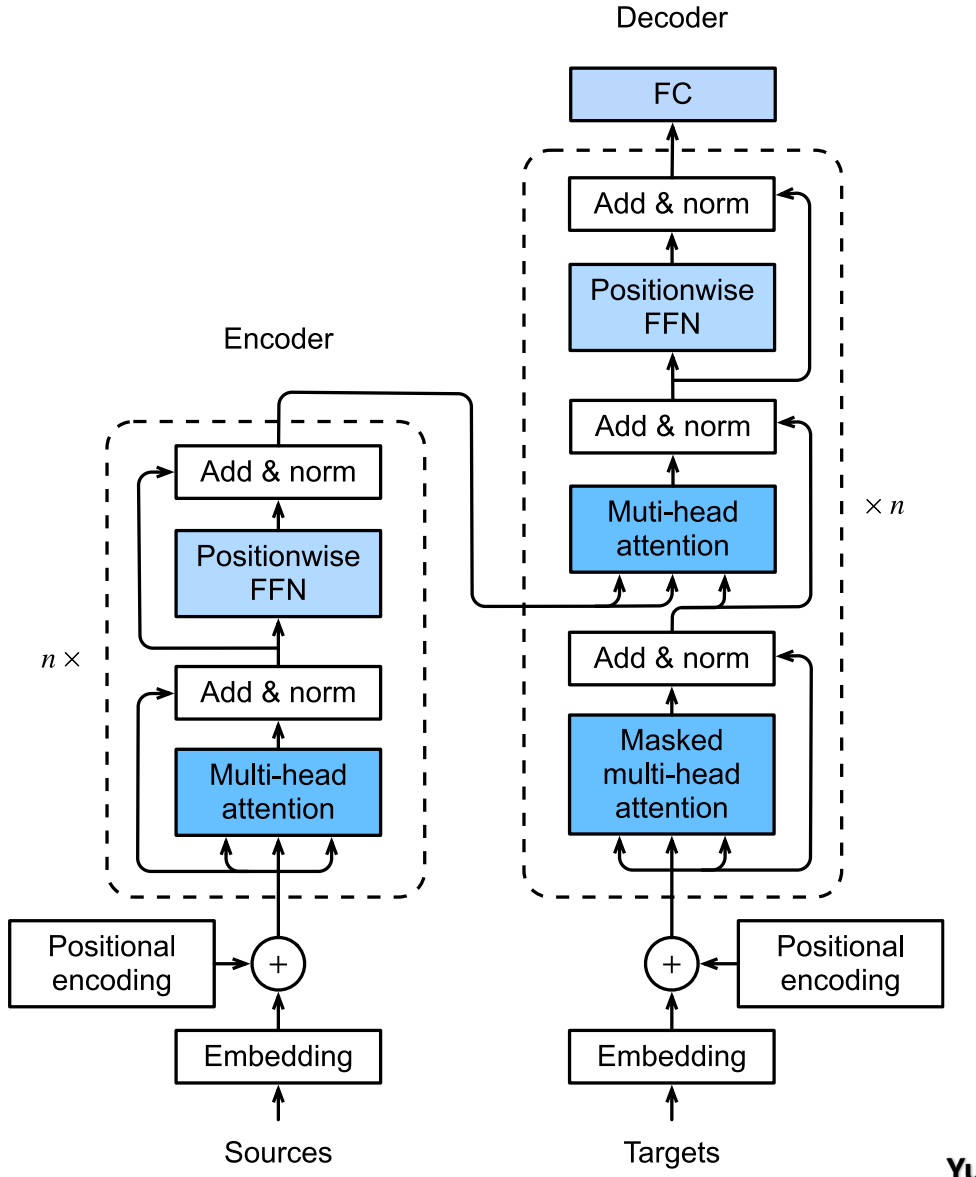


L*Hidden state dimension

L*(Hidden state dimension*n_head)

L*Hidden state dimension

Transformer for machine translation



Within each block:

Multi-head attention to learn the hidden state

FCNet to control dimension (for easy implementation)

Multiple blocks: increase complexity

Masked attention: do not consider padding position

You can understand it fully only after you have read their code and run the program

To make you awake



https://ureply.mobi/teacher

Resource and advanced topics



- Dive into deep learning: Chapter 10, 11

- Empirical evaluation of gated recurrent neural networks on sequence modeling
- Long short-term memory
- Framewise phoneme classification with bidirectional LSTM and other neural network architectures
- Sequence to sequence learning with neural networks
- Learning phrase representations using RNN encoder-decoder for statistical machine translation

Post-lecture survey



- https://forms.gle/wNZkCABLTYVJowJNA

Next time

- Attention





Thank you!

Yu LI (李煜)

liyu95.com

liyu@cse.cuhk.edu.hk

The Artificial Intelligence in Healthcare (AIH) Group

Department of Computer Science and Engineering (CSE)

The Chinese University of Hong Kong (CUHK)